

Hochschule Ravensburg-Weingarten
University of Applied Sciences

Institut für Künstliche Intelligenz

Bachelorarbeit
Angewandte Informatik

Schätzung der initialen Pose eines Objekts in einer 3D-Punktwolke

Jonathan Wekesser

Betreuer	Prof. Dr. rer. nat. Markus Schneider Prof. Dr. Daniel Scherzer
Arbeit vorgelegt am	17. Oktober 2025
Anschrift des Verfassers	Goethestraße 4, 88255 Baienfurt Mail: jonathan-wekesser@web.de Tel.: +49 176 53798538

Abstract

Die präzise Bestimmung der initialen 6D-Pose eines Objekts in einer Szene stellt eine zentrale Herausforderung in der Robotik und Computer Vision dar. Ziel dieser Arbeit ist die Entwicklung und Evaluierung einer Pipeline zur Poseinitialisierung auf Basis von RGB-D-Daten und klassischen feature- sowie geometriebasierten Verfahren. Dazu werden eine Vorverarbeitung der Punktwolken, die Merkmalsextraktion mittels Fast Point Feature Histograms (FPFH), die robuste Hypothesengenerierung durch Random Sample Consensus (RANSAC) sowie die Feinregistrierung mit Iterative Closest Point (ICP) eingesetzt. Um die Robustheit der Registrierung zu erhöhen, werden zusätzlich ansichtsbasierte Teilmodelltemplates eingeführt, welche die tatsächliche Sichtbarkeit des Objekts gezielter abbilden. In systematischen Experimenten werden die Einflüsse von Sampling-Strategien, Punktdichte, Anzahl der Teilmodelltemplates sowie eine umfassende Hyperparameteroptimierung untersucht. Die Ergebnisse zeigen, dass Teilmodelltemplates insbesondere bei komplexen oder asymmetrischen Objekten die Robustheit der Registrierung verbessern. Während bei diesen komplexeren Formen mehr Templates benötigt werden, reichen bei rotationssymmetrischen Formen bereits wenige Templates aus, um die Objektgeometrie abzubilden. Darüber hinaus ermöglicht eine universelle Konfiguration stabil gute Ergebnisse über unterschiedliche Objekte hinweg. Insgesamt wird damit belegt, dass klassische Verfahren auch ohne Deep-Learning-Ansätze eine zuverlässige Initialisierung der Objekt-pose erlauben, sofern die Parameter sorgfältig abgestimmt sind und die Templates das Objekt adäquat abdecken. Die vorgestellte Methode bietet eine fundierte Grundlage für Anwendungen wie die robotische Greifplanung in strukturierten Umgebungen.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Problemstellung	2
1.3	Zielsetzung	2
1.4	Aufbau der Arbeit	3
2	Theoretische Grundlagen	4
2.1	6D-Pose eines Objekts	4
2.2	RGB-D Kameras und Tiefendaten	5
2.3	Segmentierung in der Computer Vision	7
2.4	3D-Punktwolken	7
2.5	Punktwolkenregistrierung	8
2.5.1	Featurebasierte Vorregistrierung mit FPFH	8
2.5.2	Robuste Schätzung mit RANSAC	9
2.5.3	Verfeinerung mit ICP	9
2.6	Metriken	10
2.6.1	Anzahl der Inlier	10
2.6.2	Fitness	11
2.6.3	Inlier Root Mean Squared Error (Inlier-RMSE)	11
2.6.4	Translationsfehler	11
2.6.5	Rotationsfehler	12
2.6.6	Metriken - Zusammenfassung	12
3	Methodik und Implementierung	13
3.1	Überblick über den Gesamtprozess	13
3.2	Datenaufnahme und RGB-D-Verarbeitung	15
3.3	Segmentierung und Maskierung	15
3.4	Projektion der Szene in eine Punktwolke	17
3.5	Modellpunktwolke	17
3.6	Teilmodelltemplates	19
3.7	Punktwolkenvorbereitung	22
3.8	Vorregistrierung: Initiale Transformation	23
3.9	Registrierung	23
3.9.1	Featurebasierte Registrierung	24
3.9.2	Feinregistrierung mit ICP	24
3.10	Einordnung eines Registrierungsergebnisses	24
3.11	Hyperparameteroptimierung	26
3.11.1	Suchraumdefinition	26
3.11.2	Zielmetrik	27

3.11.3	Durchführung der Parameteroptimierung	27
3.11.4	Ergebnisse der Parameteroptimierung	28
3.11.5	Erstellung einer universellen Konfiguration	28
3.12	Implementierungsdetails	29
4	Evaluierung und Experimente	31
4.1	Evaluierung der Sampling-Strategie und Punktdichte in Modellpunktswolken	31
4.1.1	Versuchsaufbau	31
4.1.2	Ergebnisse	32
4.1.3	Diskussion	32
4.2	Evaluierung der Anzahl an Teilmodelltemplates	35
4.2.1	Versuchsaufbau	35
4.2.2	Ergebnisse	35
4.2.3	Diskussion	36
4.3	Evaluierung der Universalkonfiguration	38
4.3.1	Versuchsaufbau	39
4.3.2	Ergebnisse	39
4.3.3	Diskussion	40
4.4	Evaluierung der Teilmodelltemplates gegenüber Vollmodelltemplates	40
4.4.1	Versuchsaufbau	40
4.4.2	Ergebnisse	41
4.4.3	Diskussion	44
4.5	Diskussion	46
4.5.1	Hauptergebnisse	46
4.5.2	Kritische Reflexion	47
4.5.3	Bezug zur Forschungsfrage	47
4.6	Zusammenfassung	48
4.6.1	Fazit	48
4.6.2	Ausblick	48
	Literaturverzeichnis	50
	Quellenverzeichnis	52
A	Anhang	55
A.1	Dokumentation der Verwendung von KI-Werkzeugen	55
A.2	Quellcode	56
A.3	Konfigurationen	56
A.3.1	Baseline Konfiguration	56
A.3.2	Optimierte Konfigurationen	57
A.3.3	Universalkonfiguration	57
A.4	Verteilungen der Teilmodelltemplate-Ansichten	58
A.5	Optuna Suchbereiche	59
A.6	Ergebnisse der Evaluierung der Teilmodelltemplates gegenüber Vollmodell-templates	60
A.7	Hardware- und Software-Spezifikationen	61

1 Einleitung

1.1 Motivation

Die exakte Bestimmung der 6D-Pose eines Objekts, definiert als die Kombination aus Translation und Rotation im dreidimensionalen Raum, stellt eine zentrale Herausforderung in der Robotik und Computer Vision dar. Diese Fähigkeit ist essenziell für Anwendungen wie automatisierte Montagesysteme, autonome Fahrzeuge, Augmented Reality oder die robotische Greifplanung, da sie eine robuste Interaktion zwischen Maschine und Umgebung ermöglicht [1]. Insbesondere in der Industrie, wo Präzision und Effizienz von entscheidender Relevanz sind, sowie in der Forschung, wo flexible und kostengünstige Lösungen gefragt sind, hat die 6D-Pose-Schätzung eine hohe Signifikanz.

Die Herausforderungen der 6D-Pose-Schätzung resultieren aus der Komplexität realer Szenen, die durch Okklusionen, variierende Beleuchtungsbedingungen, symmetrische Objekte oder unstrukturierte Umgebungen geprägt sind [2]. Traditionelle Ansätze zur Erfassung von 3D-Informationen, wie etwa LiDAR oder Stereo-Kameras, liefern zwar präzise Tiefendaten, sind jedoch oft kostenintensiv, hardwareabhängig und schwer in bestehende Systeme zu integrieren [3]. RGB-D-Kameras bieten in diesem Zusammenhang eine vielversprechende Alternative, da sie sowohl Farb- als auch Tiefeninformationen liefern und in zahlreichen Anwendungen bereits integriert sind. Ihre Kosteneffizienz und Kompaktheit prädestinieren sie insbesondere für den Einsatz in kontrollierten Umgebungen wie Laboratorien oder industriellen Fertigungsstätten.

Neben der Szenenerfassung unterscheiden sich die Verfahren insbesondere in der weiteren Verarbeitung. Entweder kommt ein klassischer, auf etablierten Computer-Vision-Algorithmen basierender Ansatz zum Einsatz, oder ein moderner, lernbasierter Ansatz, der neuronale Netze verwendet.

Diese Arbeit verfolgt das Ziel, die Greifplanung von Robotern zu optimieren und leistet damit einen Beitrag zum *Institut für Künstliche Intelligenz* der Hochschule Ravensburg-Weingarten.

1.2 Problemstellung

Die zentrale Forschungsfrage dieser Arbeit lautet:

Wie gut lässt sich die initiale 6D-Pose eines Objekts in einer Szene auf Basis von RGB-D-Daten durch Template-Based Matching mit klassischen, feature- und geometriebasierten Verfahren schätzen und welchen Einfluss hat die Verwendung von Teilmodelltemplates auf die Genauigkeit und Robustheit der Registrierung?

Dabei steht die Genauigkeit der Pose-Schätzung im Vordergrund; Aspekte wie Laufzeit oder Echtzeitfähigkeit werden bewusst nachrangig betrachtet. Die Registrierung erfolgt auf Basis eines bekannten 3D-Modellobjekts (Template Matching) und verwendet ausschließlich geometrie- und featurebasierte Methoden.

1.3 Zielsetzung

Die Zielsetzung dieser Arbeit besteht in der Entwicklung und prototypischen Umsetzung eines Verfahrens zur initialen Schätzung der 6D-Pose eines Objekts in einer Szene auf Basis von RGB-D-Daten, welches sich durch besondere Robustheit auszeichnet. Die Pose wird in Form einer homogenen Transformation ermittelt, wobei die Translation und Rotation des Objekts im Kamerakoordinatensystem bestimmt werden.

Der Fokus liegt auf der Nutzung klassischer Computer-Vision-Algorithmen zur Verarbeitung von Punktwolken, beispielsweise durch geometrische Merkmale (FPFH), robuste Korrespondenzfindung (RANSAC) und verfeinerte Registrierung (ICP). Auf datengetriebene neuronale Ansätze wird bewusst verzichtet, da diese sich nach den internen Erfahrungswerten des betreuenden Instituts als zu fehleranfällig gegenüber externen Einflüssen wie Beleuchtung oder Kamerawinkel erwiesen haben.

Die entwickelte Pipeline zielt darauf ab, eine möglichst präzise und stabile Initialregistrierung zu gewährleisten, wobei Echtzeitfähigkeit oder kontinuierliches Tracking nicht angestrebt werden. Das Verfahren wurde insbesondere für die robotische Greifplanung in strukturierten Umgebungen konzipiert und zeichnet sich durch seine Modularität und Sensorunabhängigkeit aus. Dadurch ist eine flexible Integration in bestehende Systeme möglich.

1.4 Aufbau der Arbeit

Die vorliegende Arbeit ist so strukturiert, dass sie die Entwicklung und Evaluation des Verfahrens zur Schätzung der initialen 6D-Pose systematisch darstellt. Zunächst werden die theoretischen Grundlagen und die verwendeten konventionellen Computer-Vision-Algorithmen vorgestellt, um ein fundiertes Verständnis der Methodik zu schaffen. Im weiteren Verlauf wird der Aufbau der entwickelten Softwarepipeline erörtert, welche die Pose-Schätzung als Ergebnis liefert. Abschließend erfolgt eine Analyse der Leistungsfähigkeit und Robustheit des Verfahrens anhand von Experimenten und deren Testergebnissen.

2 Theoretische Grundlagen

In diesem Kapitel werden die zentralen theoretischen Konzepte vorgestellt, die der in dieser Arbeit entwickelten Methode zur Schätzung der initialen Pose eines Objekts zugrunde liegen. Zu den relevanten Begriffen zählen die der 6D-Pose, die Funktionsweise von RGB-D-Kameras, die Erzeugung und Struktur von 3D-Punktwolken sowie die wesentlichen Algorithmen zur Registrierung und Verarbeitung der Punktwolken.

2.1 6D-Pose eines Objekts

Die *6D-Pose* eines Objekts beschreibt dessen vollständige Position und Orientierung im dreidimensionalen Raum. Sie setzt sich aus drei Translationsparametern (x, y, z) sowie drei Rotationsparametern (ϕ, θ, ψ) zusammen, die beispielsweise durch Euler-Winkel, Quaternionen oder eine Rotationsmatrix dargestellt werden können.

Typischerweise wird die 6D-Pose mithilfe einer homogenen Transformationsmatrix $T \in \mathbb{R}^{4 \times 4}$ beschrieben:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix}$$

wobei $R \in SO(3)$ eine Rotationsmatrix und $t \in \mathbb{R}^3$ einen Translationsvektor darstellt. Die Spezialorthogonale Gruppe in drei Dimensionen, $SO(3)$, ist definiert als

$$SO(3) = \{R \in \mathbb{R}^{3 \times 3} \mid R^\top R = I, \det(R) = +1\}.$$

Diese Definition findet sich z. B. in Hashim (2019) sowie in literaturwissenschaftlichen Übersichten zu Rotationsparametrisierungen [4]. Sie beschreibt genau die orientierungserhaltenden Rotationen im dreidimensionalen Raum.

Eine homogene Transformationsmatrix beschreibt die Pose des Objekts stets relativ zu einem übergeordneten Referenzsystem. In dieser Arbeit ist dies das Kamerakoordinatensystem. Die Menge aller solcher Transformationen bildet die sogenannte *Special Euclidean*

Group $SE(3)$, welche die orientierungserhaltenden Starrkörperbewegungen im dreidimensionalen Raum beschreibt.

Diese kompakte Darstellung erlaubt es, Translationen und Rotationen konsistent zu kombinieren und bildet die Grundlage für Verfahren zur Registrierung von Punktwolken. Zugleich wird sie in der späteren Evaluierung benötigt, um Unterschiede zwischen einer geschätzten Pose T_{est} und einer Referenzpose T_{ref} anhand von Translations- und Rotationsfehlern zu quantifizieren (vgl. [Kap. 2.6](#)).

2.2 RGB-D Kameras und Tiefendaten

RGB-D-Kameras kombinieren eine Farbkamera (RGB) mit einem Tiefensensor (Depth) und liefern für jedes Pixel sowohl einen Farbwert als auch einen Tiefenwert (vgl. [Abb. 2.1](#)). Die Tiefe ist typischerweise entlang der optischen Achse gemessen und ergibt sich je nach Sensortyp aus Strukturlicht, Time-of-Flight oder Stereotriangulation [\[5–8\]](#).

Für die Projektion eines 3D-Punktes $P = (X, Y, Z)^\top$ auf das Bild wird die intrinsische Kameramatrix K verwendet:

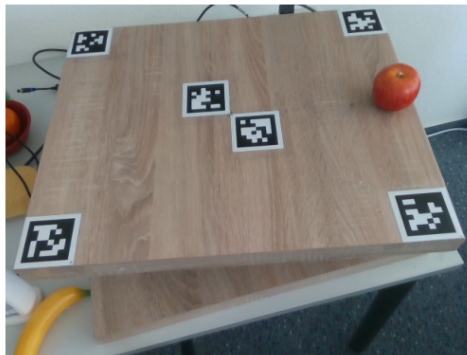
$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2.1)$$

Hierbei sind f_x, f_y die Brennweiten in Pixeln und c_x, c_y die Koordinaten des Bildmittelpunkts. Umgekehrt lässt sich mit dieser Matrix aus einem Pixelkoordinatenpaar (u, v) und einem Tiefenwert z ein 3D-Punkt rekonstruieren:

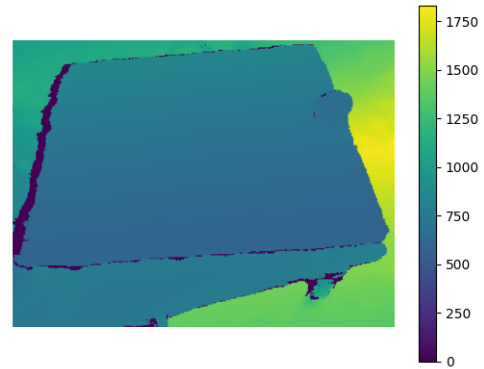
$$X = \frac{(u - c_x) \cdot z}{f_x}, \quad Y = \frac{(v - c_y) \cdot z}{f_y}, \quad Z = z \quad (2.2)$$

Dies ist notwendig um die Kamerakoordinaten in ein metrisches Koordinatensystem zu überführen, welches bei der Erstellung von Punktwolken verwendet wird.

Kameraorientierung. Für die Beschreibung der Kameraposition relativ zu einem Punkt werden die beiden Winkel *Azimuth* und *Elevation* verwendet, welche in [Abb. 2.2](#) grafisch dargestellt sind.



(a) RGB-Bild



(b) Tiefenbild

Abb. 2.1: Bild eines Apfels als RGB und Tiefenbild.

- **Azimut** (ϕ) bezeichnet die horizontale Drehung der Kamera um die vertikale Achse. Ein Azimutwinkel von 0° entspricht einer Frontalansicht, positive Winkel beschreiben eine Drehung nach links (gegen den Uhrzeigersinn in der Draufsicht), negative Winkel die entgegengesetzter Richtung.
- **Elevation** (θ) beschreibt die vertikale Neigung der Kamera relativ zur horizontalen Ebene. Ein Elevationswinkel von 0° entspricht einer Ansicht in Augenhöhe, positive Winkel einer Aufsicht und negative Winkel einer Untersicht.

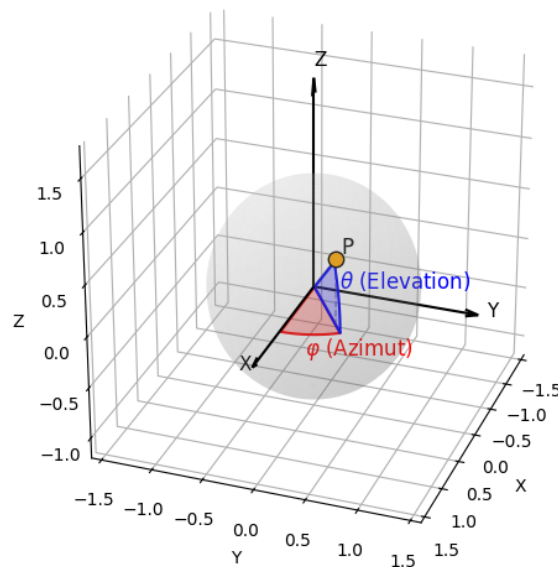
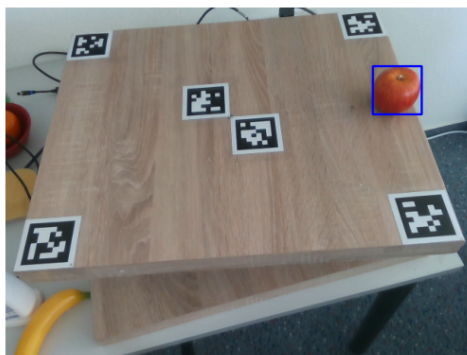


Abb. 2.2: Azimut und Elevation

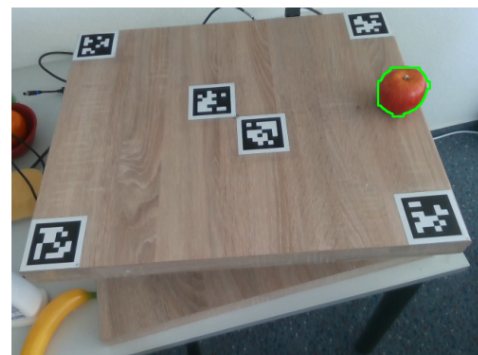
2.3 Segmentierung in der Computer Vision

Die Segmentierung ist ein grundlegender Verarbeitungsschritt in der Computer Vision, bei dem ein Bild in mehrere Regionen unterteilt wird. Ziel ist es, einzelne Objekte, Objektteile oder semantische Regionen zu identifizieren, dass sie für nachfolgende Verarbeitungsschritte wie Erkennung, Klassifikation oder Registrierung separat behandelt werden können.

Die einfachste Form der Segmentierung ist die Erstellung einer Bounding Box. Bounding Boxen sind Rechtecke oder Quader, welche um die zu betrachtende Zielregion gelegt werden, je nachdem ob es sich um einen zweidimensionalen oder dreidimensionalen Raum handelt. Der Nachteil dieser Segmentierungs-Art ist, dass zusätzlich zur eigentlichen Region ein Teil mit eingegrenzt werden kann, der nicht dazugehört. Für die Verarbeitung von Punktwolken ist dies ungeeignet, da diese Lösung viele Ausreißer erzeugen kann. Daher wird eine Segmentierung auf Punktebene vorgenommen, was mithilfe einer Maske um die Zielregion erreicht wird. Diese Methode ist genauer, jedoch ebenfalls nicht fehlerfrei. In [Abb. 2.3](#) sind die beiden Segmentierungs-Methoden dargestellt und es ist deutlich erkennbar, dass die Punkt-Segmentierung deutlich genauer funktioniert.



(a) Bounding Box



(b) Segment

Abb. 2.3: Apfel in Bounding Box vs. Segment

2.4 3D-Punktwolken

Eine Punktwolke ist eine Menge von diskreten Punkten im dreidimensionalen Raum, die typischerweise aus Messungen eines 3D-Scanners oder einer RGB-D-Kamera resultiert. Jeder Punkt beschreibt die Position (X, Y, Z) eines Objektteils in Welt- oder Kamerakoordinaten. Punktwolken können zusätzliche Informationen wie Farbdaten oder Normalenvektoren enthalten.

Folgend dienen Punktwolken sowohl als Repräsentation des Szenenausschnitts, welcher der gescannten Umgebung entspricht, als auch des Modells von dem zu lokalisierenden Objekt.

2.5 Punktwolkenregistrierung

Ziel der Punktwolkenregistrierung ist es, zwei unterschiedliche Punktwolken, eine Quellwolke P und eine Zielwolke Q , durch eine geometrische Transformation T so aufeinander abzubilden, dass die räumliche Übereinstimmung maximiert wird. Diese Transformation wird üblicherweise als homogene Matrix dargestellt und beschreibt Translation und Rotation im dreidimensionalen Raum. Die in dieser Arbeit eingesetzte Registrierungsstrategie kombiniert drei aufeinander abgestimmte Verfahren: FPFH ermöglicht eine robuste Merkmalsextraktion und liefert initiale Korrespondenzen, RANSAC sichert durch die Identifikation konsistenter Hypothesen die Robustheit gegenüber Ausreißern, und ICP verfeinert die Transformation zur präzisen Anpassung der Punktwolken. Durch diese sequenzielle Kopplung wird eine hohe Genauigkeit bei gleichzeitig erhöhter Stabilität der Registrierung erreicht.

2.5.1 Featurebasierte Vorregistrierung mit FPFH

Die *Fast Point Feature Histograms* (FPFH) wurden von Rusu et al. eingeführt [9] und stellen deskriptive Merkmale dar, die für jeden Punkt lokale geometrische Eigenschaften erfassen. Sie basieren auf Histogrammen von Winkelrelationen zwischen Normalenvektoren in einem definierten Nachbarschaftsbereich und sind rotationsinvariant. Diese Merkmale dienen dazu, aus den Punktwolken P und Q potenzielle Korrespondenzen zwischen ähnlichen lokalen Strukturen zu identifizieren.

Der FPFH-Deskriptor für einen Punkt p_q ergibt sich aus seinem eigenen *Simplified Point Feature Histogram* (SPFH) sowie einer gewichteten Summe der SPFHs benachbarter Punkte:

$$FPFH(p_q) = SPFH(p_q) + \frac{1}{k} \sum_{i=1}^k \frac{1}{\omega_i} \cdot SPFH(p_i) \quad (2.3)$$

Dabei bezeichnet k die Anzahl der Nachbarn und ω_i den Abstand zwischen p_q und seinem Nachbarn p_i . Die resultierenden Deskriptoren dienen als Grundlage für die Initialisierung der Registrierung mittels RANSAC.

2.5.2 Robuste Schätzung mit RANSAC

Da die FPFH-basierten Korrespondenzen fehlerbehaftet sein können, etwa durch symmetrische Strukturen oder verrauschte Daten, wird das *Random Sample Consensus (RANSAC)*-Verfahren eingesetzt, um robuste Transformationen zu bestimmen [10]. Es handelt sich um einen iterativen Schätzalgorithmus, der wiederholt zufällige Teilmengen der Korrespondenzen auswählt, daraus Transformationen berechnet und die Anzahl der sogenannten Inlier (Punktpaare mit geringer Abweichung) bestimmt.

Die Transformationshypothese mit der höchsten Inlier-Anzahl gilt als beste Näherung und wird für die nachfolgende Verfeinerung verwendet. Damit dient RANSAC als Filter gegen Ausreißer und fehlerhafte Korrespondenzen.

2.5.3 Verfeinerung mit ICP

Zur weiteren Verbesserung der Registrierung wird die durch RANSAC bestimmte Transformation T mittels *Iterative Closest Point (ICP)*-Verfahren verfeinert [11]. Das Ziel ist, den mittleren euklidischen Abstand zwischen korrespondierenden Punkten der beiden Punktwolken zu minimieren. In jeder Iteration werden die nächsten Punktpaare bestimmt, anschließend eine Transformation berechnet und angewendet, bis Konvergenz eintritt oder ein Abbruchkriterium erfüllt ist.

Eine Variante ist das klassische *Point-to-Point ICP* (p2p), bei dem die Fehlermetrik auf den Abständen zwischen Punktpositionen basiert:

$$T^* = \arg \min_T \sum_{i=1}^N \|T \cdot p_i - q_i\|^2 \quad (2.4)$$

wobei p_i ein Punkt aus der Quellwolke P und q_i der nächstgelegene Punkt aus der Zielwolke Q ist. Diese Methode setzt eine gute Initialisierung voraus – wie sie durch FPFH und RANSAC gewährleistet wird – da diese sonst in lokalen Minima konvergieren kann.

Alternativ kann *Point-to-Plane ICP* (p2l) eingesetzt werden, das zusätzlich die Oberflächennormalen der Zielpunkte berücksichtigt:

$$e^k = \sum_{i=1}^N d_s^2(T \circ T^{k-1} p_i, S_i^k) \quad (2.5)$$

wobei

$T \circ T^{k-1} = T^k,$	
$S_j^k = \{s \mid n_{q_j^k} \cdot (q_j^k - s) = 0\}$	die Tangentialebene an Q bei q_j^k ,
$n_{q_j^k}$	die Normale an der Oberfläche Q bei q_j^k ,
$q_j^k = (T^k \ell_i) \cap Q$	der Schnittpunkt von Q mit der Geraden $T^k \ell_i$,
$\ell_i = \{a \mid (p_i - a) \times n_{p_i} = 0\}$	die Normale an P im Punkt p_i ,
$p_i \in P$	ein Punkt aus der Quellwolke P ,
d_s	der signierte Abstand von einem Punkt zu einer Ebene.

Diese Variante führt häufig zu einer schnelleren und stabileren Konvergenz, insbesondere bei ebenen oder flächigen Strukturen [12].

In den Optimierungsergebnissen (vgl. [Kap. 3.11](#)) erweist sich die Verwendung von Point-to-Point-ICP als genauer als Point-to-Plane-ICP.

2.6 Metriken

Die Bewertung der Qualität einer geschätzten Pose in einer 3D-Punktwolke erfordert die Definition geeigneter Metriken. Diese Metriken dienen dazu, die Übereinstimmung zwischen einer geschätzten Transformation T_{est} und einer bekannten Referenzpose T_{gt} zu quantifizieren. Im Folgenden werden die in dieser Arbeit verwendeten Kennzahlen vorgestellt.

2.6.1 Anzahl der Inlier

Die *Anzahl der Inlier* (N_{inlier}) gibt die absolute Zahl der Punktkorrespondenzen an, die innerhalb des definierten Schwellwerts als gültig akzeptiert werden. Sie liefert somit ein direktes Maß für die Menge an unterstützenden Korrespondenzen einer Registrierung. Während die Fitness diese Zahl relativ zur Quellpunktwolke setzt, erlaubt die absolute Inlier-Anzahl eine Einschätzung der Robustheit der Registrierung im Hinblick auf die Datenbasis: Eine hohe Anzahl an Inliern erhöht die Zuverlässigkeit der Schätzung, auch wenn die Fitness oder der RMSE ähnliche Werte aufweisen.

2.6.2 Fitness

Die *Fitness* beschreibt das Verhältnis der Inlier-Korrespondenzen zu allen möglichen Korrespondenzen. Eine hohe Fitness deutet darauf hin, dass ein signifikanter Anteil der Punkte der Quellpunktswolke in einem vorgegebenen Schwellwertbereich mit Punkten der Zielpunktswolke übereinstimmt. Formal definiert gilt:

$$\text{Fitness} = \frac{N_{\text{inlier}}}{N_{\text{source}}} \quad (2.6)$$

wobei N_{inlier} die Anzahl der Inlier-Korrespondenzen und N_{source} die Anzahl der Punkte in der Quellpunktswolke bezeichnet. Eine niedrige Fitness weist in der Regel auf eine unzureichende Registrierung oder auf starke Unterschiede zwischen den Punktswolken hin.

2.6.3 Inlier Root Mean Squared Error (Inlier-RMSE)

Der *Inlier-RMSE* ist das mittlere quadratische Fehlermaß der Inlier-Korrespondenzen und beschreibt die durchschnittliche Abweichung zwischen den gematchten Punktpaaren. Die Einheit ist, entsprechend des Kamerakoordinatensystems, in Metern. Dieser wird wie folgt berechnet:

$$\text{Inlier-RMSE} = \sqrt{\frac{1}{N_{\text{inlier}}} \sum_{i=1}^{N_{\text{inlier}}} \|p_i - q_i\|^2} \quad (2.7)$$

mit p_i und q_i als korrespondierenden Punkten aus Quell- bzw. Zielpunktswolke. Ein niedriger RMSE bedeutet eine engere Übereinstimmung der Punktswolken im Inlier-Bereich.

2.6.4 Translationsfehler

Der Translationsfehler e_t quantifiziert die euklidische Distanz zwischen der geschätzten und der wahren Translation. Gegeben seien t_{est} und t_{gt} die Translationsvektoren von T_{est} und T_{gt} :

$$e_t = \|t_{\text{est}} - t_{\text{gt}}\|_2 \quad (2.8)$$

Der Wert wird in Metern angegeben und beschreibt den räumlichen Versatz des Objekts.

2.6.5 Rotationsfehler

Der Rotationsfehler e_r ergibt sich aus der Relativrotation $\Delta R = R_{\text{est}}^\top R_{\text{gt}}$, welche die Orientierung beschreibt, die die geschätzte Rotation R_{est} in die wahre Rotation R_{gt} überführt. Für eine Drehung um den Winkel θ gilt $\text{tr}(\Delta R) = 1 + 2\cos(\theta)$, woraus sich der spur-basierte Ansatz ableiten lässt [13]:

$$e_r = \arccos\left(\frac{\text{tr}(R_{\text{est}}^\top R_{\text{gt}}) - 1}{2}\right).$$

Der resultierende Winkel wird in Grad angegeben und repräsentiert die Orientierungsdifferenz zwischen beiden Transformationen.

2.6.6 Metriken - Zusammenfassung

Die vorgestellten Metriken ergänzen sich gegenseitig: Während Fitness und RMSE die Qualität der Punktwolkenüberlagerung bewerten, geben Translations- und Rotationsfehler direkte Auskunft über die Abweichung der geschätzten Pose von der Referenzpose. Eine ganzheitliche Bewertung der Registrierung sollte stets mehrere dieser Kennzahlen berücksichtigen, da eine einzelne Metrik allein nicht immer die gesamte Güte des Ergebnisses widerspiegelt. Zu beachten ist jedoch, dass in einer realen Anwendung nur Fitness und RMSE zur Verfügung stehen, da die Translationsfehler und Rotationsfehler nur mithilfe der Referenzpose erkannt werden können.

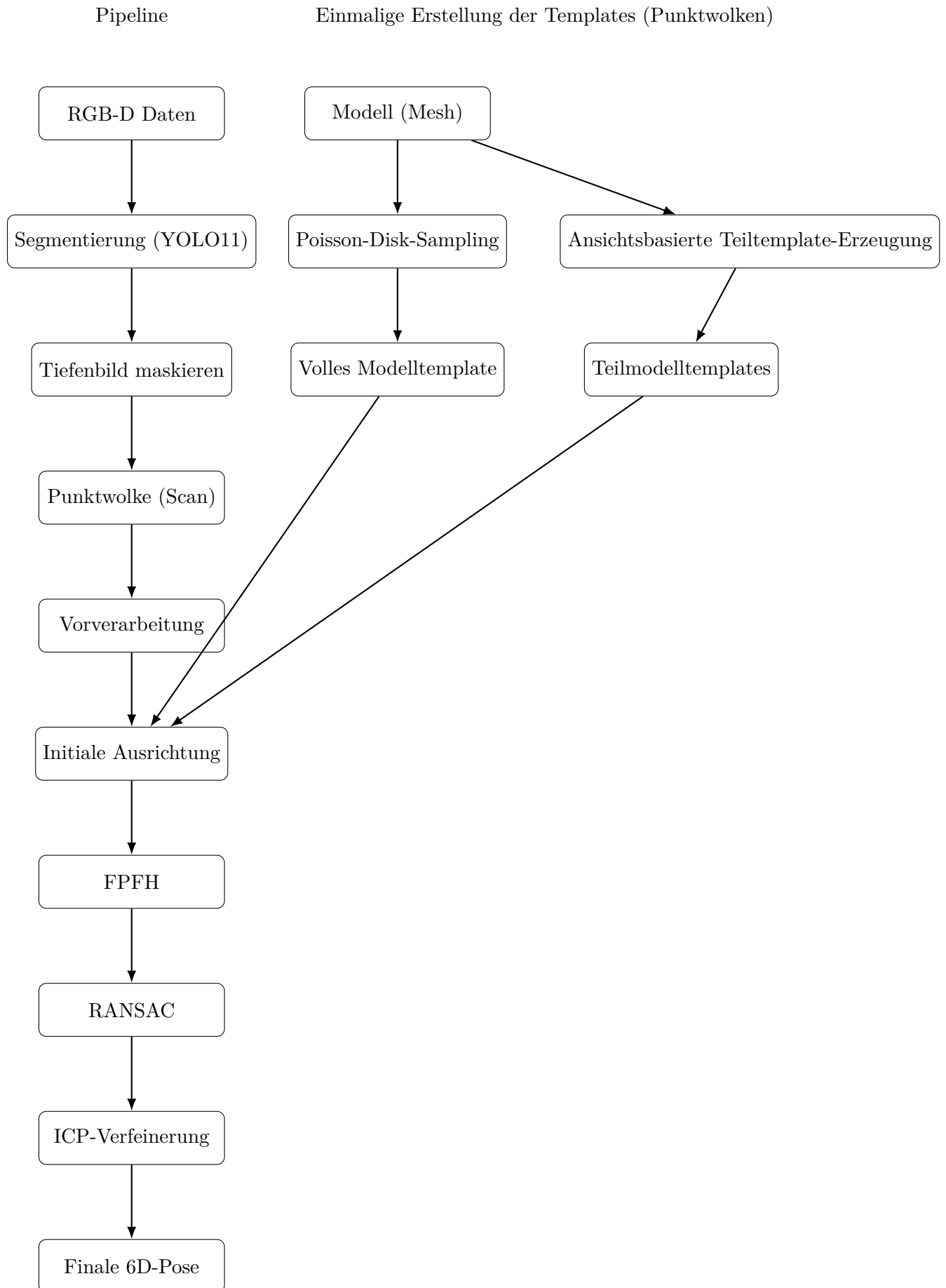
3 Methodik und Implementierung

Zur Schätzung der initialen Pose wurde ein mehrstufiges Verfahren entwickelt, das in diesem Kapitel methodisch erläutert wird. Im Rahmen dessen erfolgt eine detaillierte Vorstellung der funktionalen Module sowie eine Dokumentation ihrer technischen Implementierung.

3.1 Überblick über den Gesamtprozess

Die initiale Pose-Schätzung erfolgt in mehreren aufeinander folgenden Schritten. Der Prozess der Verarbeitung, dargestellt in [Abb. 3.1](#), initiiert mit der Erfassung der RGB-D-Daten, welche mittels einer Tiefenkamera erfasst werden (s. [Kap. 3.2](#)). Im vorliegenden Kontext wird aus dem RGB-Bild der Kamera durch ein Segmentierungsmodell eine Maske für den gesuchten Gegenstand erstellt (s. [Kap. 3.3](#)). Im Anschluss wird diese Maske auf das Tiefenbild angewandt, sodass die Punktwolke der Szene generiert wird (s. [Kap. 3.4](#)). Die unbearbeitete Punktwolke wird für die anschließenden Mapping-Algorithmen vorbereitet. Das Modell, welches im Mesh-Format vorliegt, muss ebenfalls in eine Punktwolke konvertiert werden, was jedoch lediglich einmal durchzuführen ist, da die Daten anschließend dauerhaft gespeichert werden, um den Rechenprozess effizienter zu gestalten (s. [Kap. 3.5](#)). Im darauffolgenden Schritt werden die Punktwolke des Modells und die der Szene überlagert, um eine initiale Orientierung für die nachfolgenden Algorithmen zu generieren (s. [Kap. 3.7](#)). Die grobe Ausrichtung wird mithilfe von RANSAC gelöst (s. [Kap. 2.5.2](#)). Dafür werden die deskriptiven Merkmale der Punktwolken benötigt, welche zuvor durch FPFH errechnet werden (s. [Kap. 2.5.1](#)). Im letzten Schritt wird die Feinabstimmung durch den ICP-Algorithmus durchgeführt (s. [Kap. 2.5.3](#)).

Das Resultat ist eine Schätzung der Pose des gesuchten Objekts.

**Abb. 3.1:** Verarbeitungspipeline zur initialen 6D-Pose-Schätzung

3.2 Datenaufnahme und RGB-D-Verarbeitung

Die Datengrundlage bilden synchronisierte RGB- und Tiefenbilder, die mit einer RGB-D-Kamera aufgenommen wurden. Die Tiefenbilder liegen als Graustufen-Numpy-Arrays im Format $(H \times W)$ vor. Die Farb- und Tiefenbilder werden für die jeweilige Objektinstanz geladen und in ein gemeinsames Format überführt.

Um die zuverlässige Konvertierung vom RGB-D-Bild zur Punktwolke zu gewährleisten, wird die Kamera-Intrinsik benötigt, welche bei jedem Kameramodell anders ist. In dieser Arbeit wurde das Modell Intel® RealSense™ Tiefenkamera D435i [14] verwendet (s. Tab. 3.1). Diese Kamera gehört zur Familie der Stereo-Kameras, welche die sogenannte Stereo-Image-Sensing-Technologie verwenden. Das bedeutet, dass zwei Kameras eingesetzt werden, um die Tiefe zu ermitteln und es somit erlauben, die Umgebung als 3D-Bild aufzunehmen [15].

Tab. 3.1: Kameraintrinsik für Intel Realsense D435i

Parameter	Wert
Bildbreite, -höhe	640 px, 480 px
Brennweite (f_x, f_y)	607.27 px, 607.32 px
Hauptpunkt (c_x, c_y)	318.99 px, 248.74 px

Die für diese Arbeit verwendeten RGB-D-Szenen stammen aus dem hochschulinternen Projekt *RoboTeach* am Institut für Künstliche Intelligenz der Hochschule Ravensburg-Weingarten. Die Datenbasis umfasst synchronisierte Farb- und Tiefenbilder verschiedener Objekte in strukturierten Szenen.

3.3 Segmentierung und Maskierung

Zur Extraktion des Zielobjekts aus dem RGB-Bild wird ein vortrainiertes YOLO11-Modell der Ultralytics-Bibliothek verwendet. Die Modelle dieser Bibliothek unterscheiden sich in Bezug auf Genauigkeit, Laufzeit und Komplexität (vgl. Tab. 3.2). Die Daten in der Tabelle stammen von Ultralytics und basieren auf mit dem COCO-Datensatz vortrainierten Modellen. Die Ergebnisse wurden auf COCO val2017 validiert.

Zur Validierung auf die Daten für diese Arbeit wird eine Evaluierung auf 507 Apfel- und 501 Bananenbildern durchgeführt. Dies sind die Anzahlen der jeweils zur Verfügung stehenden Datensätze. Dazu werden die fünf Segmentierungsmodelle der Ultralytics-Bibliothek *YOLO11* eingesetzt. Die Ergebnisse sind in Tab. 3.3 zusammengefasst, welche

Tab. 3.2: Ultralytics YOLO11 Segmentation Modelle

Modell	mAPmask	Geschwindigkeit (ms)	params (M)	FLOPs (B)
YOLO11n-seg	32.0	65.9 ± 1.1	2.9	10.4
YOLO11s-seg	37.8	117.6 ± 4.9	10.1	35.5
YOLO11m-seg	41.5	281.6 ± 1.2	22.4	123.3
YOLO11l-seg	42.9	344.2 ± 3.2	27.6	142.2
YOLO11x-seg	43.8	664.5 ± 3.2	62.1	319.0

Hinweis: Es handelt sich um einen Auszug der von Ultralytics veröffentlichten Ergebnisse. Die vollständige Tabelle ist in der Originalquelle einsehbar (vgl. Quellverzeichnis 4.6.2).

auch den erwarteten Zielkonflikt der Segmentierung darstellt. Große Modelle (*YOLO11l/x-seg*) erreichen über 97 % Genauigkeit, sind jedoch deutlich langsamer (0.2–0.3 s/Bild). Das kleinste Modell (*YOLO11n-seg*) ist mit ca. 85 % Genauigkeit wesentlich effizienter (0.03 s/Bild).

Tab. 3.3: Ergebnisse der Segmentierungsevaluierung mit YOLO11-Modellen.

Modell	Objekt	Gefunden/Gesamt	Genauigkeit [%]	Zeit/Bild [s]
YOLO11n-seg	Apfel	433 / 507	85.4	0.03
	Banane	432 / 501	86.2	0.03
YOLO11s-seg	Apfel	473 / 507	93.3	0.06
	Banane	477 / 501	95.2	0.07
YOLO11m-seg	Apfel	432 / 507	85.2	0.17
	Banane	492 / 501	98.2	0.17
YOLO11l-seg	Apfel	493 / 507	97.2	0.20
	Banane	492 / 501	98.2	0.20
YOLO11x-seg	Apfel	497 / 507	98.0	0.37
	Banane	493 / 501	98.4	0.38

Da die Segmentierung nur als vorbereitender Schritt für die Registrierung dient, wird *YOLO11n-seg* verwendet. Es bietet bei sehr guter Laufzeit eine hinreichende Genauigkeit, sodass es für diese Arbeit geeignet ist.

Die von dem Segmentierungsmodell erkannten Objekte werden in Form von Konturmasken bereitgestellt. Diese Masken werden als binäres Bild $M \in \{0,1\}^{H \times W}$ dargestellt:

$$M(u,v) = \begin{cases} 1 & \text{falls Pixel } (u,v) \text{ zur Objektmaske gehört} \\ 0 & \text{sonst.} \end{cases} \quad (3.1)$$

Durch Multiplikation mit dem Tiefenbild wird der Hintergrund entfernt, sodass eine reduzierte Szene entsteht, die nur Punkte innerhalb der Objektkontur enthält.

3.4 Projektion der Szene in eine Punktwolke

Das maskierte Tiefenbild wird mithilfe der bekannten Kameraintrinsik in eine metrische 3D-Punktwolke konvertiert. Für jedes gültige Pixel (u, v) mit zugehörigem Tiefenwert z wird ein 3D-Punkt X, Y, Z im Kamerakoordinatensystem berechnet (vgl. Gl. 2.2).

Die optische Achse liegt dabei in Z-Richtung, der Ursprung befindet sich im Kamerazentrum. Zur Verbesserung der Datenqualität und Reduktion von Störungen wird eine statistische Ausreißerentfernung durchgeführt. Dabei werden Punkte entfernt, deren durchschnittlicher Abstand zu den k nächsten Nachbarn signifikant vom Mittelwert abweicht. Dies reduziert das Risiko von Fehlregistrierungen. Zusätzlich erfolgt eine Schätzung der Normalenvektoren. Diese sind erforderlich für die spätere Feature-Extraktion mit FPFH (s. Kap. 2.5.1) und ermöglichen eine robustere Registrierung der Punktwolken.

3.5 Modellpunktwolke

Das Referenzobjekt liegt als trianguliertes 3D-Mesh im STL-Format vor (vgl. Abb. 3.2). Um eine Registrierung mit der gescannten Szene durchführen zu können, wird das Mesh in eine diskrete Punktwolke überführt. Dies ist notwendig, da die im weiteren Verlauf eingesetzten Verfahren zur Registrierung auf Punktwolken basieren. Für die Konvertierung existieren verschiedene Sampling-Methoden, wie *Uniform Sampling* oder *Poisson Disk Sampling*. Letztere Methode stellt sicher, dass zwischen den Punkten ein Mindestabstand eingehalten wird, was zu einer homogenen und lücken-ärmeren Verteilung führt (vgl. Abb. 3.3).

Die Wahl in dieser Arbeit fällt anhand des Experiments in Kap. 4.1 auf *Poisson Disk Sampling* mit 10 000 Punkten.

Die resultierende Punktwolke bildet die Oberfläche des Objekts gleichmäßig ab und enthält Normalenvektoren, die für die Feature-Extraktion erforderlich sind.

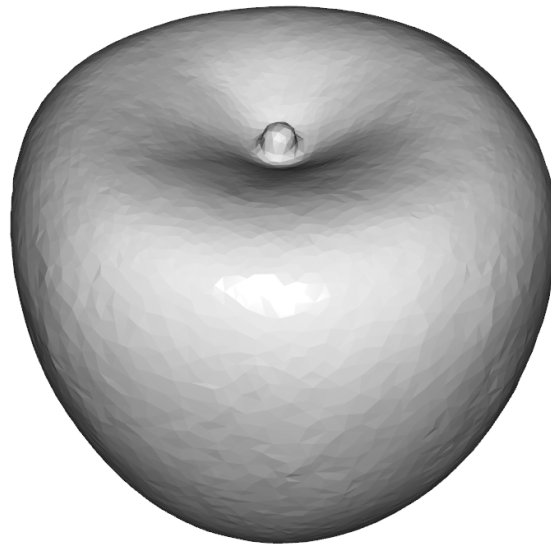
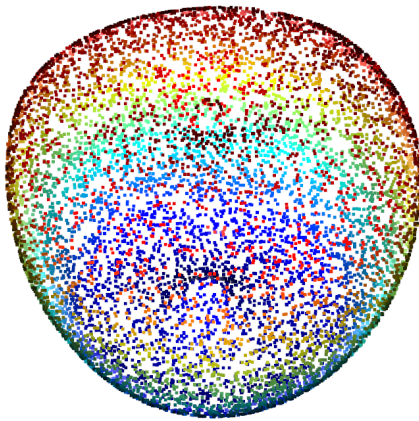
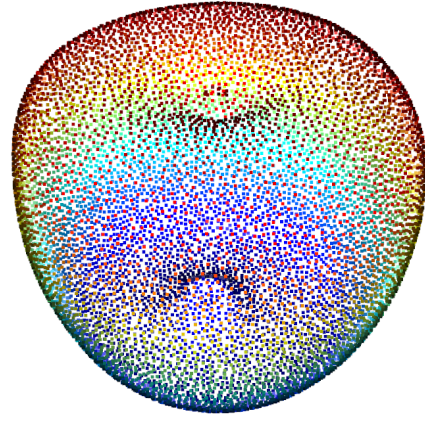


Abb. 3.2: Modell des Apfels im Mesh-Format



(a) Sample Uniformly



(b) Sample Poisson Disk

Abb. 3.3: Modell des Apfels als Punktwolke mit 10 000 Punkten

Die für die Registrierung benötigten 3D-Modelle der Objekte stammen aus dem YCB-Objektdatensatz [16]. Dieser Datensatz stellt standardisierte Mesh-Modelle und reale Objekte bereit, die in der Robotikforschung weit verbreitet eingesetzt werden, um eine vergleichbare Evaluierung von Greif- und Manipulationsaufgaben zu ermöglichen. Es werden stellvertretend die Objekte Apfel (*013_apple*) und Banane (*011_banana*) verwendet. In der Evaluierung werden zusätzlich die Objekte Orange (*017_orange*) und eine Flasche für Bleichmittelreiniger (*021_bleach_cleanser*) verwendet.

3.6 Teilmodelltemplates

Die Herausforderung bei der Verwendung einer Modellpunktvolke liegt darin, dass diese das gesamte Modell abbildet, während die Szene, die durch den Kamerawinkel erfasst wird, nur einen Ausschnitt des Objekts darstellt. Dies führt zu einer signifikanten Diskrepanz in der Anzahl der Punkte sowie der sichtbaren Oberfläche zwischen der Modellpunktvolke und der Szenenpunktvolke. Um diesem Problem entgegenzuwirken, werden Teilmodelltemplates aus dem Originalmodell generiert. Diese Templates repräsentieren spezifische Ausschnitte des Modells aus verschiedenen Perspektiven. Für jedes Template wird eine Registrierung durchgeführt, und das Template mit dem besten Registrierungsergebnis (vgl. Kap. 3.10), wird ausgewählt. Durch Experimente wird die benötigte Anzahl an Templates zur Abbildung des Modells ermittelt (vgl. Kap. 4.2) und es wird bestätigt, dass die Verwendung von Teilmodelltemplates eine Verbesserung der Registrierung zur Folge hat (vgl. Kap. 4.4).

Die Generierung der Teilmodelltemplates erfolgt durch das Rendern des 3D-Modells aus verschiedenen Kameraperspektiven, die systematisch ausgewählt werden, um die Geometrie des Objekts möglichst umfassend abzudecken. Anschließend werden die gerenderten Ansichten in Punktwolken überführt.

Für ein rotationssymmetrisches Objekt reicht eine stark begrenzte Anzahl von Templates aus, um die relevanten Ansichten zu repräsentieren.

Für einen Apfel wird Rotationssymmetrie um die Vertikalachse angenommen (Stiel oben und Boden unten). Daher genügt pro Höhenring (Elevation) jeweils ein Template; die Varianz entsteht ausschließlich über unterschiedliche Elevationswinkel. Um die Sichtkugel flächenmäßig gleich zu belegen, wird der sogenannte *Equal-Area-Ansatz* verwendet. Dabei werden die Ansichtspunkte zunächst gleichmäßig im Sinusraum verteilt und anschließend in Winkel rücktransformiert. Für $n \geq 2$ mit Winkelgrenze $\Theta = 85^\circ$ gilt:

$$s_i = \sin(-\Theta) + \frac{i-1}{n-1} (\sin(\Theta) - \sin(-\Theta)), \quad i = 1, \dots, n, \quad (3.2)$$

$$el_i = \arcsin(s_i). \quad (3.3)$$

Hierbei bezeichnet n die Anzahl der Höhenringe (Elevationen), s_i den äquidistanten Stützpunkt im Sinusraum und el_i den daraus resultierenden Elevationswinkel in Grad. Der Parameter Θ legt die maximale Elevation fest, die in dieser Arbeit auf 85° gesetzt wird, um numerische Instabilitäten an den Polen zu vermeiden.

Für $n = 1$ wird $el_1 = 0^\circ$ gewählt (Äquatorring). Die Wahl $Theta = 85^\circ$ vermeidet numerische Degenerierungen an den Polen ($\pm 90^\circ$); *forward* fast parallel zu *up* und reduziert leere Raycast-Treffer.

Die Banane hingegen ist nicht symmetrisch, was bedeutet, dass mehr Templates erstellt werden müssen um diese abzubilden.

In diesem Fall werden für den Apfel Templates mit $n = 5$; $az = 0$; $el \in \{-85, -30, 0, 30, 85\}$ (s. Abb. 3.4) ausgewählt. Für die Banane sind es deutlich mehr Templates mit $n = 25$; $az \in \{0, 72, 144, 216, 288\}$; $el \in \{-60, -30, 0, 30, 60\}$ (s. Abb. 3.5).

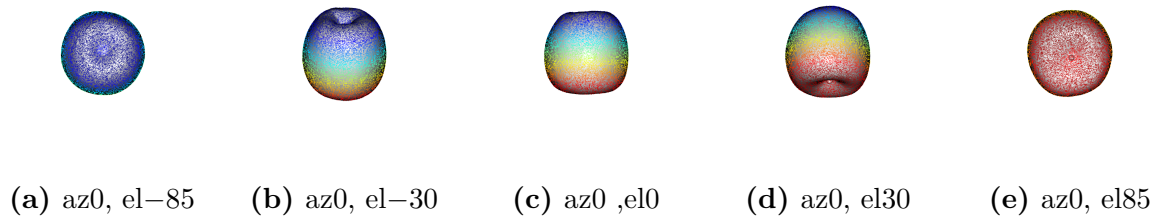
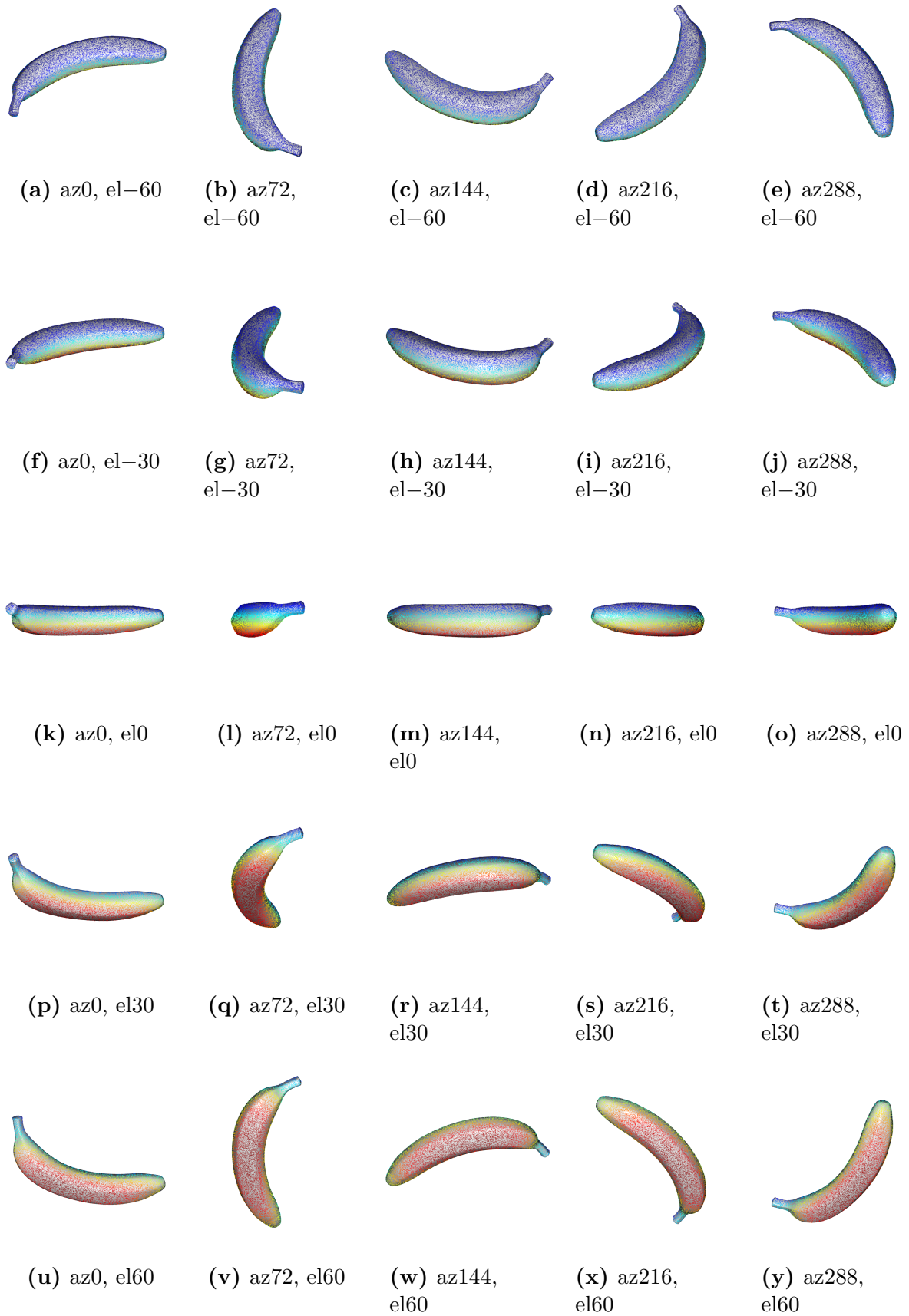


Abb. 3.4: Apfel Teilmodelltemplates mit $n = 5$

Abb. 3.5: Bananen Teiltemplates mit $n = 25$

3.7 Punktwolkenvorbereitung

Ziel der Punktwolkenvorbereitung ist es, Rauschen und Ausreißer zu reduzieren, die Datenmenge zu begrenzen und stabile lokale Geometrieinformationen (Normalen) für die nachfolgenden Schritte (FPFH, RANSAC, ICP) bereitzustellen. In der Implementierung erfolgt dies in drei Schritten: *(i)* Ausreißerentfernung, *(ii)* Voxel-Downsampling und *(iii)* Normalenschätzung.

(i) Ausreißerentfernung. Zur Robustheitssteigerung gegenüber Messrauschen und Fehlsegmentierungen wird zunächst ein statistischer Ausreißerfilter angewandt. Für jeden Punkt wird der mittlere Abstand zu seinen k nächsten Nachbarn bestimmt; Punkte, deren mittlerer Abstand mehr als das in den Parametern definierte Vielfache der Standardabweichung über dem globalen Mittelwert liegt, werden entfernt. Dadurch werden isolierte Punkte in der Szenepunktwolke und Artefakte am Maskenrand zuverlässig unterdrückt.

(ii) Voxel-Downsampling. Zur Kontrolle der Laufzeit und zur Harmonisierung des Abstandsmaßstabs wird die Punktwolke mit einer Voxelgröße unterabgetastet. Das Downsampling reduziert die Punktzahl (und damit die Komplexität aller nachfolgenden Nachbarschaftssuchen) und stabilisiert lokale Merkmale, da kleinskaliges Sensorrauschen gemittelt wird. Wichtig ist, dass nachfolgende Radiusparameter (etwa für FPFH und Normalen) konsistent zur definierten Voxelgröße gewählt werden, um genügend Nachbarn pro Abfrage zu erhalten (vgl. [Kap. 2.5.1](#)).

(iii) Normalenschätzung. Viele Verfahren wie FPFH oder ICP in der Point-to-Plane-Variante setzen die Verfügbarkeit von Oberflächennormalen voraus. Diese werden in einer Hybrid-Nachbarschaft über k-d-Baum-Abfragen geschätzt, die durch einen Suchradius und eine maximale Nachbarschaftsgröße parametrisiert ist. Ein zu kleiner Radius führt zu verrauschten und instabilen Normalen, während ein zu großer Radius Kanten glättet und die Diskriminativität lokaler Merkmale verringert. In der hier verwendeten Pipeline wird der Normalenradius proportional zur Voxelgröße gewählt, sodass sich über verschiedene Auflösungen hinweg eine konsistente Nachbarschaftsdichte ergibt. Während für FPFH konsistente Normalen zwingende Voraussetzung sind, werden sie bei ICP nur für die Point-to-Plane-Variante benötigt.

Insgesamt stellen diese Schritte sicher, dass (i) Ausreißer die Korrespondenzsuche nicht dominieren, (ii) Laufzeit und Speicher planbar bleiben, und (iii) die für FPFH und ICP benötigten lokalen Flächennormalen mit ausreichender Stabilität vorliegen.

3.8 Vorregistrierung: Initiale Transformation

Vor der eigentlichen Registrierung wird eine grobe Vorregistrierung durchgeführt, um die räumliche Lage des Modells näherungsweise an die gescannte Szene anzugleichen. Ziel ist es, die nachfolgenden Algorithmen zu entlasten, indem der Suchraum für die Transformation verkleinert wird und initiale Fehlanpassungen vermieden werden. Hierzu werden die Schwerpunkte der beiden Punktwolken berechnet, definiert als arithmetisches Mittel aller 3D-Koordinaten der jeweiligen Punktmenge:

$$c = \frac{1}{N} \sum_{i=1}^N p_i \quad (3.4)$$

Die Modellpunktwolke wird anschließend durch eine reine Translation verschoben, sodass ihr Schwerpunkt mit dem der Szene übereinstimmt. Die Transformationsmatrix T_{init} ist dabei wie folgt definiert:

$$T_{\text{init}} = \begin{bmatrix} I_{3 \times 3} & c_{\text{scan}} - c_{\text{modell}} \\ 0 & 1 \end{bmatrix} \quad (3.5)$$

wobei c_{scan} und c_{modell} die Schwerpunktkoordinaten der Scan- und Modellpunktwolke darstellen. Diese Transformation wird vor der Anwendung von FPFH und RANSAC auf die Modellpunktwolke angewendet.

Da keine Rotation berücksichtigt wird, ist die Transformation nicht ausreichend für eine exakte Ausrichtung. Sie dient lediglich der Groborientierung und stellt sicher, dass sich beide Punktwolken in räumlicher Nähe befinden, was eine Voraussetzung für das effiziente Funktionieren der featurebasierten Registrierung darstellt.

3.9 Registrierung

Die Registrierung selbst besteht aus zwei Schritten. Der groben Registrierung, welche featurebasiert durchgeführt wird, und der Feinregistrierung.

3.9.1 Featurebasierte Registrierung

Für die featurebasierte Registrierung werden die Verfahren FPFH (s. [Kap. 2.5.1](#)) und RANSAC (s. [Kap. 2.5.2](#)) verwendet. Mithilfe von FPFH werden die Merkmale (Features) berechnet. Diese werden für das Matching mithilfe von *Random Sample Consensus* (RANSAC) verwendet. Dieser Algorithmus kommt zum Einsatz, da die entstehenden Korrespondenzen typischerweise auch fehlerhafte Zuordnungen enthalten und dieser robust damit umgehen kann. Dabei werden Kandidatentransformationen generiert und anhand von geometrischen Konsistenzkriterien bewertet. Nur Inlier-Korrespondenzen, die die Distanzschwellen und Randbedingungen erfüllen, fließen in die Schätzung der Transformation ein.

Das Ergebnis dieser Phase ist eine grobe Ausrichtung des Modells auf die Szene. Diese Vorregistrierung bildet die Grundlage für die nachfolgende Feinregistrierung mit ICP, die in [Kap. 3.9.2](#) beschrieben wird.

3.9.2 Feinregistrierung mit ICP

Die finale Pose-Schätzung erfolgt durch eine Verfeinerung mittels *Iterative Closest Point* (ICP). Die zuvor durch RANSAC angenäherte Transformation dient als Initialisierung. ICP minimiert den Abstand zwischen korrespondierenden Punkten der beiden Punktwolken iterativ:

$$\min_T \sum_i \|T \cdot p_i - q_i\|^2 \quad (3.6)$$

wobei p_i und q_i korrespondierende Punkte der Quell- bzw. Zielwolke sind.

3.10 Einordnung eines Registrierungsergebnisses

Die Auswertung eines Registrierungsergebnisses erfolgt auf Grundlage der in [Kap. 2.6](#) beschriebenen Kennzahlen. Jede Registrierung liefert neben der geschätzten Transformation T_{est} eine Menge an Korrespondenzen sowie Gütemaße, die in Relation zur bekannten Referenzpose T_{gt} gesetzt werden können. Die Einordnung des Ergebnisses umfasst daher sowohl qualitätsbezogene als auch effizienzbezogene Aspekte.

Qualitative Bewertung. Die Kennzahlen *Fitness*, *Inlier-RMSE* und die absolute *Inlier-Anzahl* charakterisieren die Übereinstimmung der beiden Punktwolken unabhängig von einer bekannten Referenz. Eine hohe *Fitness* bei gleichzeitig niedrigem *Inlier-RMSE* deutet auf eine zuverlässige Überlagerung hin. Die *Inlier-Anzahl* ergänzt diese Einschätzung, da sie die Robustheit der Registrierung gegenüber Ausreißern widerspiegelt. Stehen zusätzlich Referenzposen zur Verfügung, können *Translationsfehler* und *Rotationsfehler* berechnet werden, die eine direkte Bewertung der Posegenauigkeit ermöglichen. Diese Metriken sind insbesondere für die Validierung im Rahmen kontrollierter Experimente entscheidend.

Effizienzbewertung. Die *Laufzeit* stellt ein ergänzendes Kriterium dar, das die Berechnungsdauer einer Registrierung beschreibt. Sie erlaubt Rückschlüsse auf die praktische Anwendbarkeit der Methode, insbesondere wenn viele Szenen oder Templates ausgewertet werden. Ein geringerer Zeitaufwand ist dabei vorteilhaft, sofern die Genauigkeit nicht signifikant beeinträchtigt wird. Die Laufzeit dient somit in dieser Arbeit primär dem Vergleich unterschiedlicher Konfigurationen und Registrierungsstrategien.

Gesamteinordnung. Ein Registrierungsergebnis gilt in dieser Arbeit als erfolgreich, wenn die *Fitness* einen hohen Wert aufweist und der *Inlier-RMSE* im Bereich weniger Millimeter liegt. Die endgültige Bewertung berücksichtigt daher stets mehrere Metriken gleichzeitig, da eine einzelne Kennzahl keine hinreichende Aussagekraft besitzt. Dieses Vorgehen ermöglicht eine differenzierte Einordnung, die sowohl die Genauigkeit als auch die Effizienz der Registrierung berücksichtigt.

Entscheidungsregel. Die Auswahl des besseren Registrierungsergebnisses erfolgt anhand einer festen Entscheidungslogik. Gegeben seien zwei Evaluationen *A* und *B* mit den Kennzahlen *Fitness*, *Inlier-RMSE* und *Inlier-Anzahl*. Die Entscheidung erfolgt wie folgt:

1. Falls *A* gleichzeitig eine höhere *Fitness* und einen geringeren *Inlier-RMSE* als *B* aufweist, wird *A* bevorzugt. Umgekehrt wird *B* bevorzugt, wenn die *Fitness* niedriger und der *Inlier-RMSE* höher ist.
2. Andernfalls wird geprüft, ob ein niedrigerer *Inlier-RMSE* mit einer größeren *Inlier-Anzahl* kombiniert ist. In diesem Fall wird die entsprechende Registrierung bevorzugt.
3. Wenn auch hier keine Entscheidung möglich ist, wird die Registrierung mit der größeren *Inlier-Anzahl* gewählt.

4. Bleiben die Ergebnisse gleichwertig, so wird die aktuelle Registrierung (hier: A) ausgewählt.

Diese Entscheidungsregel bildet die implementierte Funktion ab und stellt sicher, dass bei konkurrierenden Ergebnissen stets ein deterministisches Auswahlverfahren greift.

3.11 Hyperparameteroptimierung

Die zuvor beschriebenen Verfahren zur Registrierung von Punktwolken (Feature-Extraktion mittels FPFH, robuste Transformation mittels RANSAC sowie Feinregistrierung durch ICP) aus [Kap. 3.9](#), hängen stark von einer Vielzahl von Hyperparametern ab. Dazu zählen unter anderem die Größe des Voxel-Filters zur Down-Sampling-Strategie, die Nachbarschaftsradien für Normalenschätzung und Feature-Berechnung sowie Schwellenwerte und Iterationszahlen in den RANSAC- und ICP-Verfahren. Eine manuelle Abstimmung dieser Parameter ist aufwändig und liefert nicht zwangsläufig eine optimale Konfiguration. Aus diesem Grund wird eine systematische Hyperparameteroptimierung durchgeführt.

3.11.1 Suchraumdefinition

Für die Hyperparameteroptimierung werden die relevanten Parameterbereiche heuristisch eingeschränkt, um die Rechenzeit einzuschränken und den Suchraum auf realistische Werte einzugrenzen. Dabei sind insbesondere jene Parameter berücksichtigt, die einen maßgeblichen Einfluss auf die Registrierungsergebnisse haben.

Die Voxelgröße bestimmt die Dichte der Punktwolke nach dem Downsampling und beeinflusst unmittelbar die Robustheit und Genauigkeit der weiteren Verarbeitung. Für das Outlier Removal wird die Nachbarschaftsgröße sowie die Standardabweichungsschwelle optimiert, da diese die Balance zwischen Rauschentfernung und Informationsverlust steuern.

Für die Normalenschätzung sind der Suchradius sowie die maximale Anzahl berücksichtigter Nachbarn entscheidend. Analog gilt dies für die Featureberechnung, wo der Radius und die maximale Nachbarschaftsgröße die Qualität der FPFH-Deskriptoren beeinflussen.

Auf Ebene der robusten Vorregistrierung mittels RANSAC werden mehrere Parameter einbezogen: die maximale Korrespondenzdistanz, die Stichprobengröße, die Kantenlängenprüfung, die maximale Anzahl an Iterationen, sowie das Konfidenzniveau. Diese Parameter bestimmen, wie tolerant oder restriktiv die Hypothesenprüfung erfolgt.

Für die abschließende Feinregistrierung mit ICP werden die Varianten Punkt-zu-Punkt-ICP und Punkt-zu-Ebene verglichen, sowie die Iterationszahl optimiert. Diese Parameter steuern die Konvergenzgeschwindigkeit und Genauigkeit des Algorithmus.

Die vollständige Übersicht der gewählten Suchbereiche ist im Anhang unter [Tab. A.7](#) dargestellt.

3.11.2 Zielmetrik

Da die Registrierung auf mehreren Gütekriterien beruht (Fitness, Inlier-RMSE, Translations- und Rotationsfehler), wird eine kombinierte Bewertungsfunktion herangezogen. In diese fließen der Median der Fitnesswerte sowie die Mittelwerte der Fehlermaße ein. Zur Gewichtung werden die Fehlerarten skaliert, sodass Translationsfehler (in m) stärker und Rotationsfehler (in °) schwächer berücksichtigt werden. Dies hat den Hintergrund, dass die Rotation bei Rotationssymmetrischen Objekten, wie dem Apfel, keine große Rolle spielen. Die Zielfunktion lautet:

$$\text{Score} = \frac{\text{median}(\text{Fitness}) \cdot \text{SuccessRate}}{1 + \text{RMSE} + \alpha \cdot T_{\text{err}} + \beta \cdot R_{\text{err}}} \quad \text{mit} \quad \alpha = 10, \beta = \frac{1}{900}. \quad (3.7)$$

Die *SuccessRate* ist der Anteil der Szenen mit mindestens einer gefundenen Korrespondenz (`correspondence_set_size > 0`). Wenn keine gültigen Metriken vorliegen (z. B. NaN) oder die *SuccessRate* = 0 ist, wird der Versuch als Fehlschlag mit einem großen negativen Score (-10^6) gewertet, um solche Konfigurationen zuverlässig auszusortieren.

3.11.3 Durchführung der Parameteroptimierung

Es werden zwei Optimierungen mit jeweils 250 Datensätzen durchgeführt. Eine mit dem Objekt *Apfel* und eine mit dem Objekt *Banane*. Jede Optimierung wird mit einer festen Anzahl von $n = 20$ Versuchen pro Studie durchgeführt. Für jeden Versuch gilt ein Zeitlimit von 30 min, um sowohl die Rechenzeit handhabbar zu halten, als auch einen Abbruch bei ausbleibender Konvergenz sicherzustellen. Als Suchalgorithmus kommt der *Tree-structured Parzen Estimator* (TPE)-Sampler von Optuna zum Einsatz, der durch eine initiale Zufallssampling-Phase ergänzt wird. Auf diese Weise kann eine balancierte Exploration des Suchraums gewährleistet und gleichzeitig eine gezielte Ausnutzung vielversprechender Parameterregionen erreicht werden.

3.11.4 Ergebnisse der Parameteroptimierung

Die Optimierung wird für die Objekte *Apfel* und *Banane* auf gleich großen Datenabschnitten ($id \in [0, 249]$) durchgeführt. Als Vorlagen dienen jeweils das volle Modell und Teilmplates (*templates_used: both*). Die Zielfunktion ist in [Kap. 3.11.2](#) definiert (vgl. [Gl. 3.7](#)). Zusätzlich wird auf Szenenebene bei konkurrierenden Kandidaten die Vergleichsregel (vgl. [Kap. 3.10](#)) eingesetzt, die Lösungen mit hoher *Fitness* und niedriger *Inlier-RMSE* priorisiert.

Die Objektspezifische Optima sind im Anhang unter [Tab. A.3](#) abgebildet. Beide Objekte liegen bei der Voxelgröße nah beieinander (≈ 2 cm) und bilden damit eine sinnvolle Skala für Normalen- und FPFH-Radien. Unterschiede ergeben sich insbesondere bei den Nachbarschaftsradien und der RANSAC-Konfiguration: Beim Apfel werden kleinere Radien mit tendenziell höheren Nachbarzahlen, sowie strengere Konsistenzprüfungen bei hohem Iterationsbudget genutzt. Bei der Banane werden bevorzugt größere Radien mit weniger Nachbarn, kombiniert mit kleinerer Stichprobe und deutlich geringerem RANSAC-Budget verwendet. In der ICP-Phase arbeitet der Apfel mit einer größeren Korrespondenzschwelle und wenigen Iterationen, während die Banane eine enge Schwelle mit mehr Iterationen verwendet. Die *best_value*-Werte der Zielfunktion betragen 0.704 (Apfel) und 0.438 (Banane) und reflektieren die unterschiedlichen Geometrie- und Textur-Charakteristika.

3.11.5 Erstellung einer universellen Konfiguration

Über beide Objekte hinweg bestätigt sich eine stabile Voxelgröße im Bereich von etwa 2 cm, sodass eine Kopplung der Radien an die Voxelgröße praktikabel ist (Richtwerte: Normalenradius $\approx 2 \times$ Voxelgröße, FPFH-Radius $\approx 4 \times$ Voxelgröße). Die Banane profitiert von größeren Radien bei stärkerer Ausreißerunterdrückung, was mit ihrer elongierten Geometrie und stärkeren Selbstähnlichkeiten vereinbar ist; der Apfel toleriert strengere RANSAC-Konsistenzprüfungen bei größerem Hypothesenbudget.

Vorverarbeitung. Die Voxelgröße beträgt 0.019 m. Dieser Wert liegt nahe an den jeweiligen Objekt-Optima und reduziert die Punktzahl hinreichend, ohne relevante Geometriedetails zu verlieren. Für die statistische Ausreißerentfernung werden 95 Nachbarn und ein Standardabweichungsverhältnis von 0.16 gesetzt, wodurch sowohl dichte als auch spärliche Bereiche robust bereinigt werden.

Normalen und FPFH. Der Normalenradius ist mit 0.038 m auf etwa das Zweifache der Voxelgröße gekoppelt; bis zu 60 Nachbarn stabilisieren die Schätzung auch in heterogenen

Punktverteilungen. Für die FPFH-Merkmale wird ein Radius von 0.076 m ($\approx 4 \times$ Voxelgröße) und eine Obergrenze von 30 Nachbarn verwendet. Diese Wahl balanciert lokale Detailtreue mit der Robustheit der Korrespondenzbildung.

RANSAC (Vorangepassung). Die Korrespondenzschwelle beträgt 0.005 m. Mit einer Stichprobengröße von $r=4$, einem Kantenlängenfaktor von 0.85, maximal 60 000 Iterationen und einer Konfidenz von 0.95 ergibt sich eine robuste, gleichzeitig aber effiziente Vorregistrierung. Die Parameter harmonisieren die Tendenzen der einzelnen Objekt-Optima und vermeiden Überanpassungen an ein spezifisches Szenario.

ICP (Verfeinerung). Für die Feinregistrierung wird die Variante *Point-to-Point* mit einer Korrespondenzschwelle von 0.010 m und bis zu 120 Iterationen eingesetzt. Der Schwellenwert ist so gewählt, dass er bei kompakten und elongierten Geometrien zuverlässig konvergiert und gegenüber Tiefenrauschen moderat tolerant bleibt.

Zusammenfassung. Die Konfiguration überträgt die Stärken der objekt-spezifischen Optima auf eine robuste, allgemeine Einstellung. Die Kopplung der Radien an die Voxelgröße ($\approx 2 \times$ bzw. $\approx 4 \times$) erleichtert die Skalierung auf vergleichbare Sensorsettings. In der Kombination aus konservativer RANSAC-Absicherung und moderater ICP-Verfeinerung entsteht eine zuverlässige Initialisierung mit guter Übertragbarkeit.

3.12 Implementierungsdetails

Die gesamte Softwarearchitektur ist in der Programmiersprache Python, in der Version 3.12, umgesetzt. Ziel ist es, eine modulare und erweiterbare Umgebung zu schaffen, die eine klare Trennung zwischen Datenaufnahme, Vorverarbeitung, Registrierung und Visualisierung ermöglicht. Dies erleichtert nicht nur die schrittweise Entwicklung, sondern auch die gezielte Evaluation einzelner Verarbeitungskomponenten. Die Hard- sowie Softwarekonfiguration ist in [Kap. A.7](#) dokumentiert.

Für die Verarbeitung und Registrierung von Punktwolkendaten wurde die Bibliothek Open3D eingesetzt [17]. Sie stellt effiziente Funktionen zur Verfügung, um Punktwolken aus Tiefenbildern zu erzeugen, Normalen zu schätzen, statistische Ausreißer zu entfernen und Registrierungsmethoden wie FPFH, RANSAC oder ICP anzuwenden.

Die Segmentierung der Objekte aus den RGB-Bildern wurde mit der Ultralytics Bibliothek YOLO realisiert [18]. Sie ermöglicht es, vortrainierte YOLO-Modelle direkt für das Segmentierungsproblem zu verwenden. Die dabei erzeugten Objektmasken bilden die Grundlage für die spätere Extraktion der zugehörigen Tiefendaten.

Weitere unterstützende Aufgaben wie das Einlesen und Transformieren von Bilddaten, das Handling von Masken sowie die Konvertierung zwischen Bild- und Punktwolkendaten wurden mithilfe der Bibliotheken OpenCV und NumPy umgesetzt. Für die Visualisierung von Zwischenergebnissen kamen Matplotlib (für 2D-Bilder) sowie die interaktive Punktwolkenansicht von Open3D zum Einsatz.

Die gesamte Software wurde in logisch getrennte Module gegliedert, darunter Pfadverwaltung, Szenensteuerung, Punktwolkenoperationen und visuelle Ausgabe. Diese modulare Struktur ermöglichte es, einzelne Verarbeitungsschritte unabhängig voneinander zu entwickeln und testen, was insbesondere bei der Evaluation von Segmentierung und Registrierung von Vorteil ist.

Zur Optimierung der Hyperparameter wird das Open-Source-Framework *Optuna* eingesetzt [19]. Optuna ermöglicht eine automatisierte Suche in hochdimensionalen Parameterräumen und verwendet dabei bayesianische Optimierungsverfahren (Tree-structured Parzen Estimator, TPE) sowie optionale Pruner, die vielversprechende Suchpfade priorisieren und nicht vielversprechende frühzeitig abbrechen. Damit können komplexe Konfigurationsräume effizient durchsucht werden.

Eine vollständige Implementierung aller in diesem Kapitel beschriebenen Module sowie der gesamten Experimente-Pipeline ist im zugehörigen Repository dokumentiert (vgl. [Kap. A.2](#)).

4 Evaluierung und Experimente

In diesem Kapitel wird die entwickelte Methode zur Schätzung der initialen 6D-Pose systematisch durch Experimente evaluiert. Ziel ist es, die Leistungsfähigkeit und Robustheit des Verfahrens unter verschiedenen Bedingungen zu untersuchen und die erzielten Ergebnisse kritisch zu bewerten.

4.1 Evaluierung der Sampling-Strategie und Punktdichte in Modellpunktvolken

Wie in [Kap. 3.5](#) beschrieben, muss das Modell in eine Punktwolke überführt werden. Hierfür werden zwei Verfahren untersucht: *Uniform Sampling* und *Poisson Disk Sampling*. Ziel des Experiments ist es, die Eignung dieser Methoden im Hinblick auf die Registrierung zu bewerten und die Auswirkungen verschiedener Punktzahlen, also der Punktdichte, zu analysieren.

4.1.1 Versuchsaufbau

Als Testobjekte werden der *Apfel* und die *Banane* verwendet. Für beide Objekte werden jeweils Modellpunktvolken in unterschiedlichen Varianten erzeugt, die anschließend in 100 Datensätzen zur Registrierung eingesetzt werden.

Insgesamt stehen pro Objekt vierzig Modellvarianten zur Verfügung. Diese setzen sich aus den beiden Sampling-Verfahren *Uniform Sampling* und *Poisson Disk Sampling* mit jeweils zwanzig Varianten zusammen. Die Punktzahl wird dabei systematisch variiert: beginnend bei 1000 Punkten bis hin zu 20 000 Punkten, in Schritten von jeweils 1000.

4.1.2 Ergebnisse

Die Registrierung gelingt in allen getesteten Szenen nahezu vollständig. Für den Apfel beträgt die Erfolgsrate 100 %, für die Banane im Mittel 99.97 %.

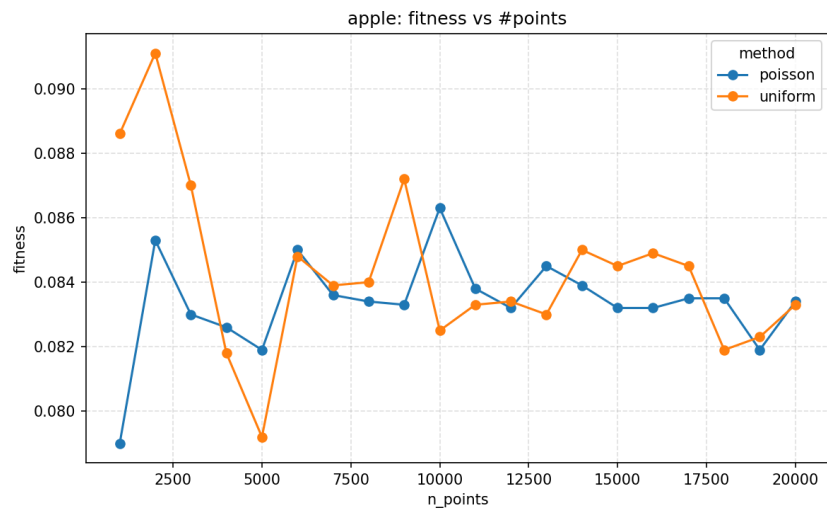
Die Fitnesswerte, welche grafisch in [Abb. 4.1](#) abgebildet sind, liegen beim Apfel zwischen 0.081 und 0.088, bei der Banane zwischen 0.049 und 0.060. Der Inlier-RMSE beträgt beim Apfel etwa 3.6 mm bis 3.7 mm, bei der Banane etwa 3.4 mm. Die Translationsfehler liegen im Bereich von 1.8 cm bis 2.1 cm für den Apfel und zwischen 3.1 cm und 3.5 cm für die Banane. Die Rotationsfehler bewegen sich beim Apfel zwischen 144° und 152° und bei der Banane zwischen 103° und 108°.

Die Laufzeit steigt mit zunehmender Punktzahl leicht an (Apfel: 0.185 s-0.238 s, Banane: 0.192 s-0.235 s), mit einer Steigung von rund 0.0016 s pro 1000 Punkte.

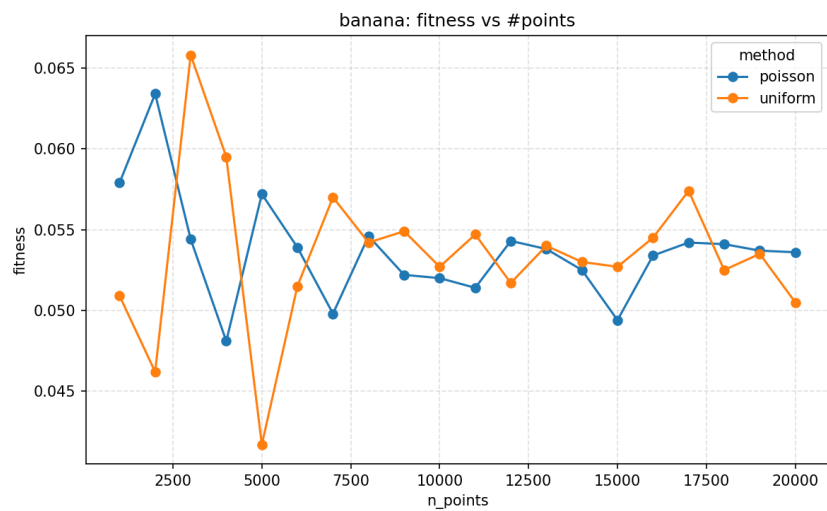
4.1.3 Diskussion

Die Ergebnisse verdeutlichen, dass die Wahl der Sampling-Strategie (Poisson vs. Uniform) keinen erkennbaren Einfluss auf die Registrierungsqualität hat. Die Fitness- und RMSE-Werte unterscheiden sich nur marginal zwischen den Methoden. Auch die Punktzahl im Bereich von 1000 bis 20 000 führt zu keinen signifikanten Qualitätsunterschieden, die Registrierung bleibt robust. Lediglich die Laufzeit steigt mit zunehmender Punktzahl leicht an (vgl. [Abb. 4.2](#)), bleibt jedoch in allen Konfigurationen unter 0.25 s pro Szene und ist damit praktisch unkritisch.

Die hohen Rotationsfehler sind durch Symmetrien der Objekte erklärbar und nicht auf Sampling-Strategie oder Punktdichte zurückzuführen. Für die folgenden Experimente wird daher eine mittlere Punktzahl von 10 000 Punkten mit Poisson-Sampling gewählt, da dieses Setting eine gleichmäßige Flächenabdeckung sicherstellt und gleichzeitig eine effiziente Verarbeitung ermöglicht.

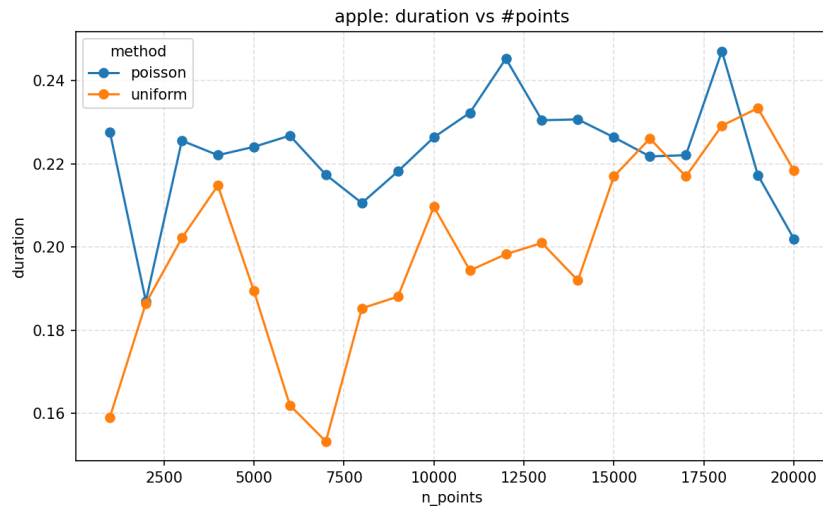


(a) Apfel

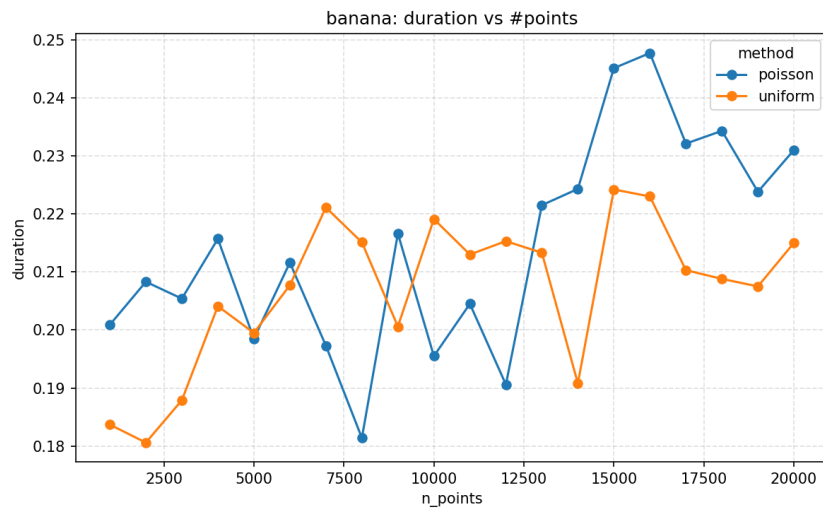


(b) Banane

Abb. 4.1: Fitness in Abhängigkeit von Sampling-Methode und Punktzahl



(a) Apfel



(b) Banane

Abb. 4.2: Durchschnittliche Laufzeit pro Szene in Abhängigkeit von Sampling-Methode und Punktzahl

4.2 Evaluierung der Anzahl an Teilmodelltemplates

In [Kap. 3.6](#) ist beschrieben, welchen Zweck die Teilmodelltemplates erfüllen und wie diese erstellt werden. Um zu ermitteln, wie viele Teilmodelltemplates ein Objekt sinnvoll abbilden, wird ein Experiment durchgeführt.

4.2.1 Versuchsaufbau

Es werden die beiden Versuchsobjekte *Apfel* und *Banane* verwendet.

Für den Apfel werden zehn Sets an Teilmodelltemplates erzeugt. Da es sich um ein rotationssymmetrisches Objekt handelt, werden alle Templates auf demselben Azimutwinkel (0°) und mit unterschiedlichen Elevationswinkeln erstellt. Die zehn Sets unterscheiden sich in der Anzahl der Templates mit $n \in \{1, \dots, 10\}$. Die Elevationsringe werden, wie im Grundlagenkapitel erläutert, *gleichflächig* (Equal-Area) verteilt (vgl. [Gl. 3.2](#) und [Gl. 3.3](#)); d. h. $\sin(\text{Elevation})$ ist äquidistant, wodurch die Kugeloberfläche pro Höhenring annähernd gleich repräsentiert wird.

Die Banane ist im Gegensatz zum Apfel nicht rotationssymmetrisch. Daher werden zusätzlich die Azimutwinkel variiert. Hierzu werden 18 verschiedene Sets an Teilmodelltemplates mit $8 \leq n \leq 30$ verglichen; die Azimutwinkel sind jeweils gleichmäßig über 360° verteilt, während die Elevationsniveaus je nach Set bei $\{0\}$, $\{\pm 15^\circ\}$, $\{\pm 30^\circ\}$, $\{\pm 45^\circ\}$ oder $\{\pm 60^\circ\}$ liegen.

Die genauen Verteilungen der Ansichten sind im Anhang unter [Tab. A.5](#) (Apfel) und [Tab. A.6](#) (Banane) dokumentiert. Alle Teilmodelltemplates werden auf 100 Datensätzen registriert.

4.2.2 Ergebnisse

Beim Apfel liegt der durchschnittliche maximale Fitnesswert ab $n = 3$ oberhalb von 0.45. Zwischen $n = 1$ und $n = 2$ zeigt sich ein deutlicher Anstieg der Fitness ($\Delta \approx 145\%$), während zwischen $n = 3$ und $n = 10$ nur noch marginale Unterschiede ($\Delta < 1\%$) erkennbar sind (vgl. [Abb. 4.3](#)). Der Median der Fitness bleibt ab $n = 2$ nahezu konstant, und auch der Inlier-RMSE variiert kaum (≈ 0.0032). Die Laufzeit steigt näherungsweise linear mit der Anzahl der Templates (vgl. [Abb. 4.4](#)).

Bei der Banane sind die Fitnesswerte insgesamt deutlich niedriger (Fitness < 0.31). Eine klare Korrelation zwischen Fitness und der Anzahl der Teilmodelltemplates lässt sich nicht

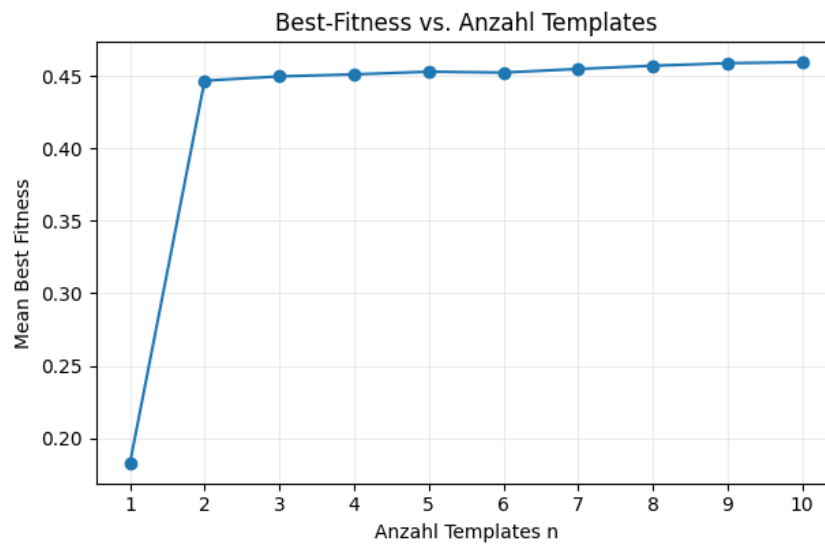


Abb. 4.3: Apfel: Durchschnittlicher maximaler Fitnesswert vs. Anzahl der Teilmodelltemplates

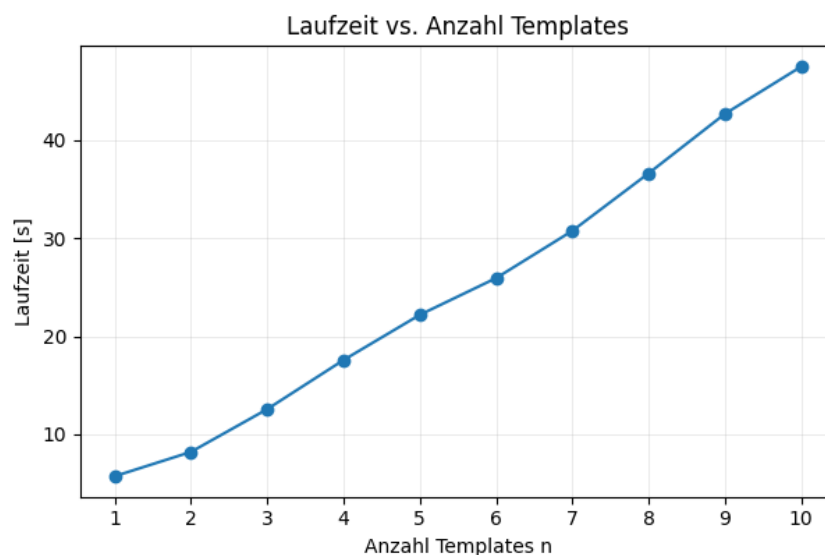


Abb. 4.4: Apfel: Laufzeit vs. Anzahl der Teilmodelltemplates

feststellen (vgl. [Abb. 4.5](#)). Die Laufzeit steigt proportional mit zunehmender Anzahl der Templates (vgl. [Abb. 4.6](#)). Der Inlier-RMSE zeigt stärkere Schwankungen (vgl. [Abb. 4.7](#)). Zur besseren Einordnung wird zusätzlich ein Trade-off-Plot zwischen Fitness und Laufzeit erstellt, der nach der Anzahl der *Höhenringe* gruppiert ist (vgl. [Abb. 4.8](#)).

4.2.3 Diskussion

Für den Apfel ist ab einer moderaten Anzahl von $n = 5$ Templates kein nennenswerter Zugewinn an Registrierungsqualität erkennbar. Zusätzliche Templates erhöhen primär die

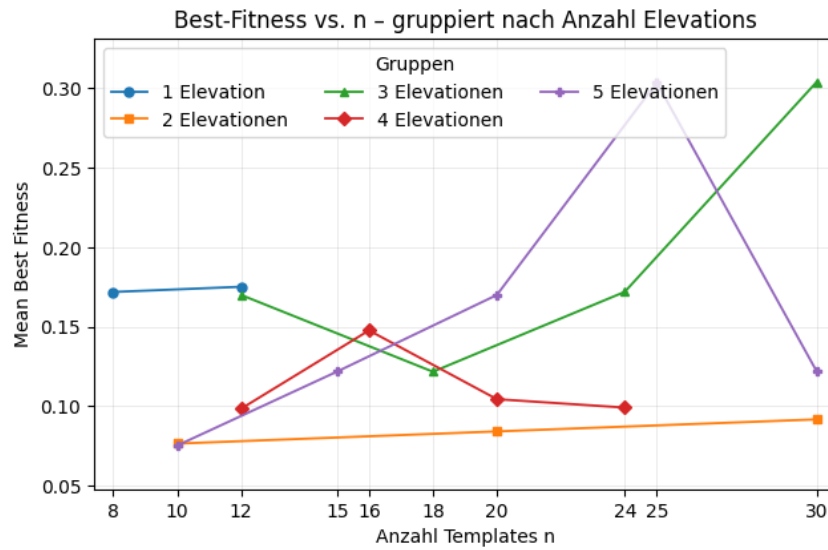


Abb. 4.5: Banane: Durchschnittlicher maximaler Fitnesswert vs. Anzahl der Teilmodelltemplates

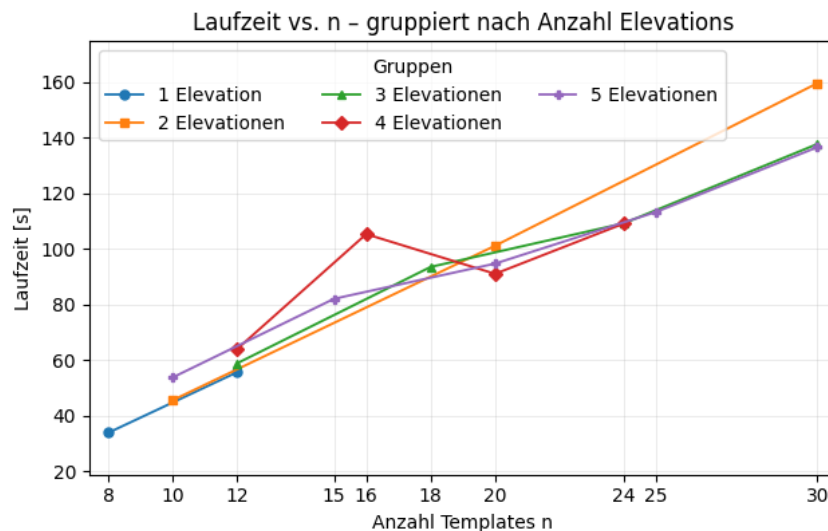


Abb. 4.6: Banane: Laufzeit vs. Anzahl der Teilmodelltemplates

Laufzeit, ohne die Fitness signifikant zu verbessern. Dies deutet darauf hin, dass die kugelförmige Geometrie des Apfels bereits mit wenigen, gleichflächig gewählten Elevationen ausreichend repräsentiert ist. Für die weiteren Experimente wird daher das Set mit fünf Teilmodelltemplates verwendet (vgl. [Abb. 3.4](#)).

Bei der Banane ist aufgrund der fehlenden Rotationssymmetrie eine feinere Abdeckung über Azimut und Elevation erforderlich. Dabei ist zwischen der Anzahl der Höhenringe und der Anzahl der Templates pro Ring abzuwägen. In den Ergebnissen erweist sich das Set mit 25 Teilmodelltemplates und fünf Höhenringen als besonders vorteilhaft: Es erreicht einen vergleichbaren Fitnesswert wie das Set mit 30 Templates auf drei Höhenringen

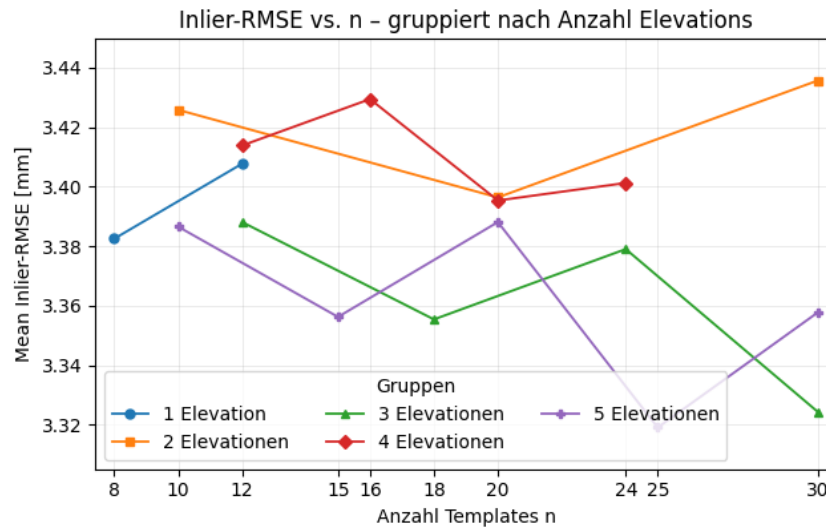


Abb. 4.7: Banane: Inlier-RMSE vs. Anzahl der Teilmodelltemplates

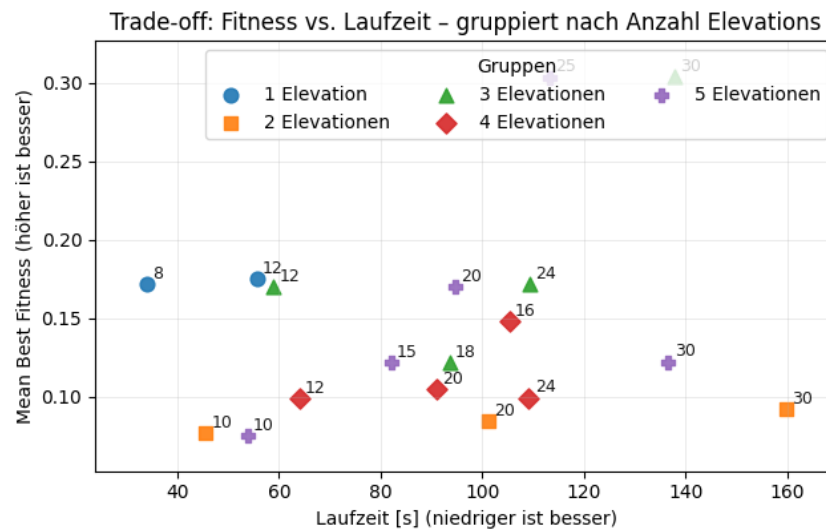


Abb. 4.8: Banane: Trade-off zwischen Fitness und Laufzeit (gruppiert nach Höhenringen)

(≈ 0.30), weist jedoch eine deutlich geringere Laufzeit auf (113 s gegenüber 137 s).

Zudem lässt sich eine proportionale Abhängigkeit der Laufzeit zur Anzahl der Teilmodelltemplates feststellen. Dies hat seinen Ursprung in dem Mehraufwand der Berechnungen pro Szene, da für jedes Teilmodell die gesamte Registrierung durchgeführt wird.

4.3 Evaluierung der Universalkonfiguration

In [Kap. 3.11](#) ist die Durchführung der Hyperparameteroptimierung beschrieben und dargestellt, wie daraus die Universalkonfiguration (vgl. [Tab. A.4](#)) abgeleitet wird. In diesem

Kapitel wird evaluiert, wie sich die Verwendung dieser Konfiguration auf die Ergebnisse der Registrierung auswirkt. Hierzu werden jeweils unabhängige Datensätze verwendet auf welche dann unterschiedliche Konfigurationen angewendet und verglichen werden.

4.3.1 Versuchsaufbau

Für die Experimente werden jeweils die Datensätzen 250 bis 349 des Apfels und der Banane verwendet. Pro Szene werden alle verfügbaren Teilmodelltemplates sowie das volle Template verwendet, wobei jeweils das beste Ergebnis (vgl. ??) behalten wird. Dafür wird für alle verfügbaren Modelltemplates eine Registrierung durchgeführt, wobei pro Szene jeweils das beste Ergebnis beibehalten wird. Für jedes Objekt werden die Baselinekonfiguration (vgl. Tab. A.2), die objektspezifisch optimierten Konfigurationen (vgl. Tab. A.3) und die Universalkonfiguration angewandt.

4.3.2 Ergebnisse

Apfel. Für den Apfel steigert die Universalkonfiguration die mittlere Fitness von 0.35 (Baseline) auf 0.95, wobei der Medianwert mit 1.0 eine nahezu perfekte Übereinstimmung in allen Szenen anzeigt. Der Inlier-RMSE erhöht sich im Vergleich zur Baseline um etwa den Faktor drei, während die Translations- und Rotationsfehler weitgehend unverändert bleiben. Die Laufzeit reduziert sich von durchschnittlich 0.044 s auf 0.026 s ($\approx -40\%$), wobei die Korrespondenzmenge von über 100 auf rund 20 Punkte absinkt. Die objektspezifischen Optimierungen (`optimized_apple`, `optimized_banana`) zeigen ein sehr ähnliches Verhalten: ebenfalls hohe Fitnesswerte, erhöhte Inlier-RMSE sowie eine reduzierte Korrespondenzanzahl. Unterschiede zwischen den Varianten und der Universalkonfiguration liegen im Detailbereich.

Banane. Auch bei der Banane verbessert die Universalkonfiguration die Fitness deutlich: der Mittelwert steigt von 0.29 (Baseline) auf 0.90, der Median erreicht 0.94. Analog zum Apfel erhöht sich der Inlier-RMSE um den Faktor drei. Die Translationsfehler bleiben im Millimeterbereich stabil, während die Rotationsfehler leicht sinken. Die Laufzeit verkürzt sich um etwa 10 %, und die Korrespondenzmenge sinkt von rund 46 auf 17. Die objektspezifischen Optimierungen erreichen vergleichbare Ergebnisse und bestätigen die gleichen Trends.

4.3.3 Diskussion

Die Ergebnisse verdeutlichen, dass die Universalkonfiguration in beiden Objekten eine deutliche Verbesserung gegenüber der Baseline darstellt. Insbesondere die Fitnesswerte erreichen nahezu perfekte Werte, während die Fehlermaße auf einem stabilen Niveau bleiben. Der Anstieg des Inlier-RMSE geht mit einer stark reduzierten Korrespondenzmenge einher und ist damit auf restriktivere Einstellungen bei der Korrespondenzsuche zurückzuführen. Die verkürzten Laufzeiten lassen sich durch die geringere Anzahl an Korrespondenzen erklären.

Die objektspezifischen Konfigurationen liefern nahezu identische Resultate. Damit zeigt sich, dass die Universalkonfiguration sowohl für den Apfel als auch für die Banane vergleichbare Verbesserungen erzielt wie die individuell optimierten Parameter. Die zentralen Effekte - höhere Fitness, erhöhter Inlier-RMSE, stabile Translations- und Rotationsfehler sowie reduzierte Laufzeit - treten bei allen Optimierungen gleichermaßen auf.

4.4 Evaluierung der Teilmodelltemplates gegenüber Vollmodelltemplates

In [Kap. 3.6](#) ist die Generierung und Verwendung von Teilmodelltemplates aus einem vollständigen Modelltemplate beschrieben. Diese Templates sollen die Registrierung in realen Szenen verbessern, da sie besser zu den tatsächlich sichtbaren Objektteilen passen. In diesem Abschnitt wird untersucht, welchen Einfluss die Verwendung von Teilmodelltemplates auf die Qualität der Posen-Initialisierung hat. Hierzu wird ein Vergleich zwischen der Registrierung mit vollständigen Modelltemplates und Teilmodelltemplates durchgeführt.

4.4.1 Versuchsaufbau

In diesem Experiment werden die Objekte *Apfel*, *Banane*, *Orange* sowie eine Flasche für Bleichmittelreiniger (im Folgenden nur noch als *Flasche* bezeichnet) verwendet. Als Konfiguration kommt die Universalkonfiguration (vgl. [Tab. A.4](#)) zum Einsatz. Für Apfel und Banane werden die zuvor entwickelten Teilmodelltemplates (s. [Kap. 3.6](#)) genutzt. Für die Orange werden, analog zum Apfel als rotationssymmetrisches Objekt, Teilmodelltemplates mit $n = 5$ erstellt. Die Flasche erhält die gleiche Verteilung wie die Banane (asymmetrisches Objekt) mit $n = 25$ Templates.

Ziel ist es, neben den beiden Hauptobjekten, auch die Generalisierbarkeit der Pipeline und der gewählten Template-Anzahlen auf weitere Objekte zu überprüfen.

4.4.2 Ergebnisse

Bei allen Objekten wird eine deutlich erhöhte Rechenzeit bei den Teilmodelltemplates gegenüber dem Vollmodelltemplate festgestellt.

Abb. 4.9 zeigt die Verteilung der Fitness-Werte für die vier untersuchten Objekte. Über alle Objekte hinweg erreicht die Verwendung von Teilmodelltemplates (*partial_best*) höhere Fitness-Werte als das Vollmodell (*full*). Die Ergebnisse des Experiments sind im Anhang unter Tab. A.8 dargestellt.

Die Inlier-RMSE (Abb. 4.10) ist bei den Teilmodelltemplates im Mittel leicht erhöht. Die Pose-bezogenen Fehler zeigen ein differenziertes Bild: Der Translationsfehler reduziert sich bei allen Objekten überwiegend, während der Rotationsfehler in ähnlicher Größenordnung verbleibt, teils jedoch größere Ausreißer aufweist. Auffällig ist hingegen, dass die Inlier-Anzahl (Abb. 4.11) im Vergleich zum Vollmodell deutlich abnimmt.

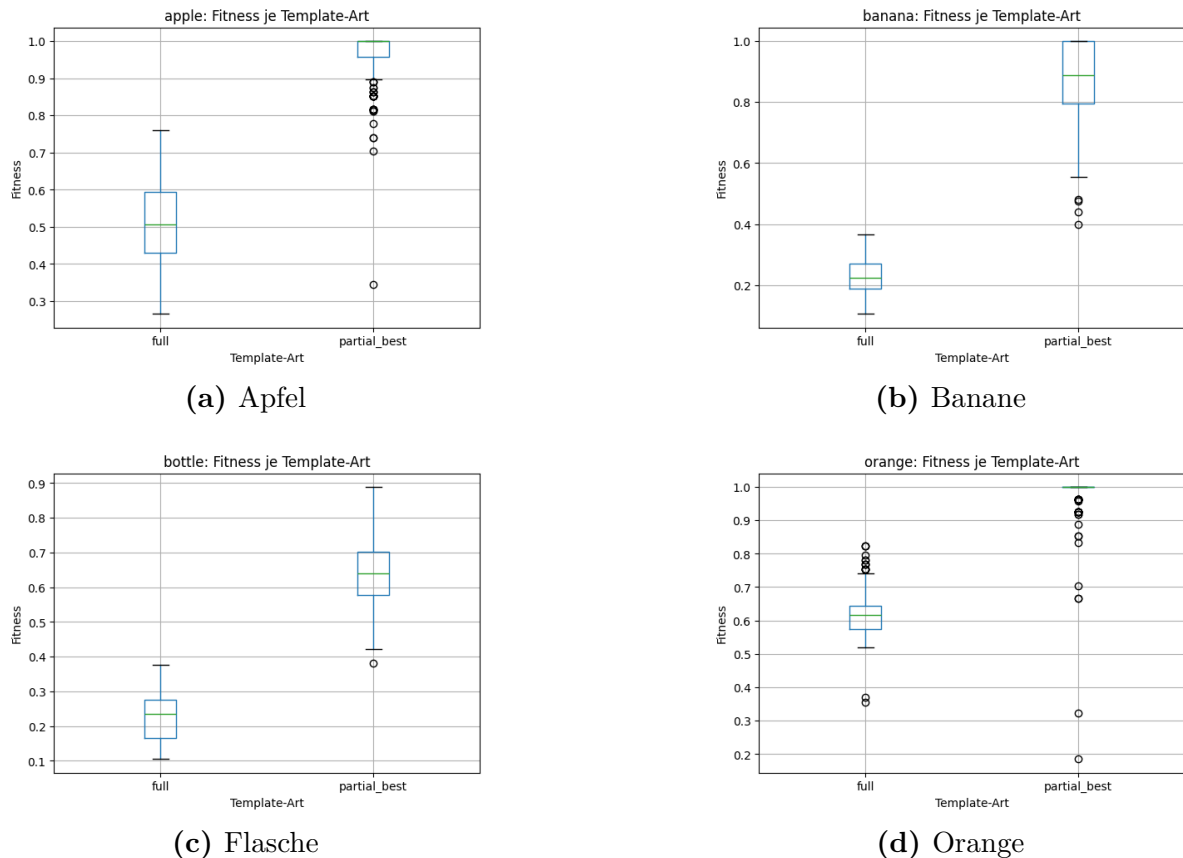
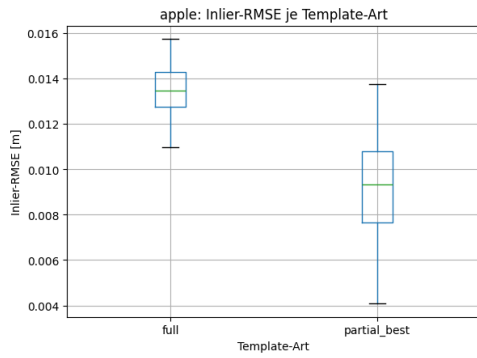
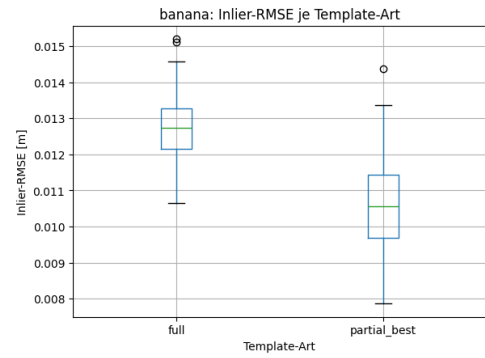


Abb. 4.9: Fitness-Verteilung Volltemplate vs. (bestes) Teilmodelltemplate



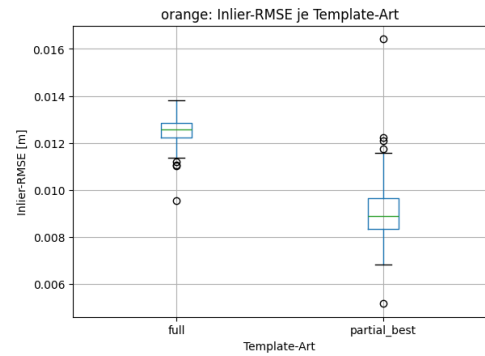
(a) Apfel



(b) Banane



(c) Flasche



(d) Orange

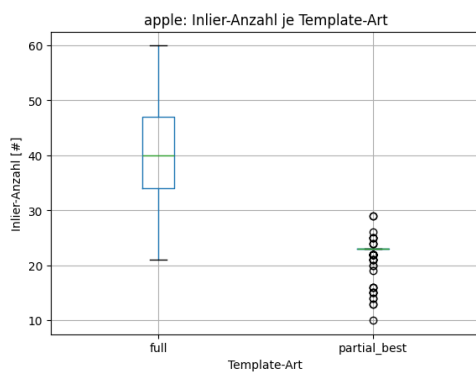
Abb. 4.10: Inlier-RMSE-Verteilung Volltemplate vs. (bestes) Teilmodelltemplate

Um die Qualität der Registrierung nachvollziehbar darzustellen, werden die Ergebnisse zusätzlich visuell aufbereitet. Dazu werden die rekonstruierte Szenenpunktwolke, das registrierte Modelltemplate sowie die ursprüngliche Tiefenpunktwolke gemeinsam dargestellt. Die einzelnen Punktwolken werden farblich unterschieden (Szene in Grün, Teilmodelltemplate in Blau, Vollmodelltemplate in Rot, Hintergrund in Schwarz), sodass sich die Lage des Modells innerhalb der Szene unmittelbar überprüfen lässt. Auf diese Weise können sowohl korrekte als auch fehlerhafte Registrierungen intuitiv beurteilt und die quantitativen Metriken (Fitness, Inlier-RMSE, Translations- und Rotationsfehler) visuell ergänzt werden.

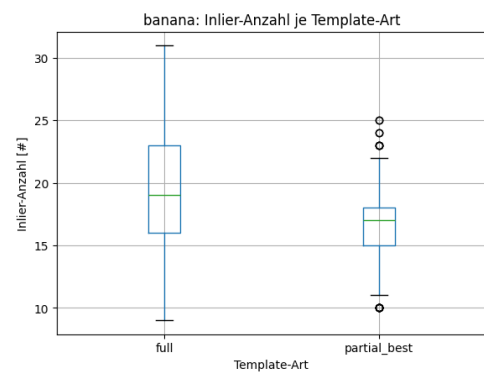
Die visuelle Analyse der Registrierungsergebnisse verdeutlicht deutliche Unterschiede zwischen den untersuchten Objekten.

Beim Apfel gelingt die Registrierung in den meisten Fällen mit guter Qualität (vgl. [Abb. 4.12](#)). Teilmodelltemplates erweisen sich hierbei als vorteilhaft, da sie die tatsächlich sichtbaren Oberflächenausschnitte genauer repräsentieren und dadurch eine präzisere Initialisierung der Pose ermöglichen.

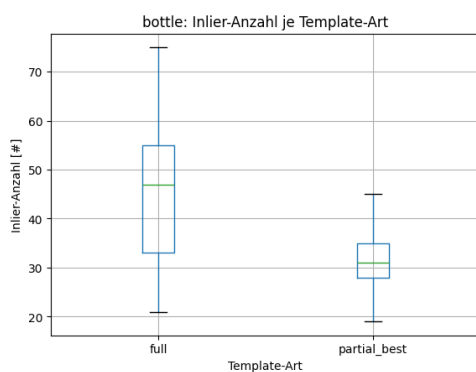
Bei der Banane fallen die Ergebnisse insgesamt schwächer aus (vgl. [Abb. 4.13](#)). Ihre elon-



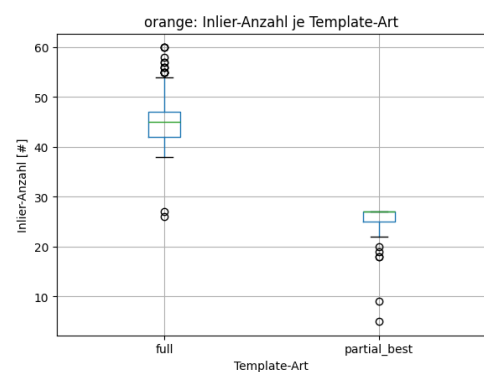
(a) Apfel



(b) Banane



(c) Flasche



(d) Orange

Abb. 4.11: Inlier-Anzahl Volltemplate vs. (bestes) Teilmodelltemplate

gierte Form und die ausgeprägte Selbstähnlichkeit entlang der Längsachse führen häufig zu Mehrdeutigkeiten, was in fehlerhaften Ausrichtungen resultiert. Unter diesen Bedingungen liefert das Vollmodelltemplate im Durchschnitt stabilere Ergebnisse als die Teilmodelltemplates.

Im Fall der Flasche ist die Registrierungsqualität ebenfalls in vielen Fällen gering (vgl. [Abb. 4.14](#)). Die rotationssymmetrische und weitgehend strukturlose Geometrie erschwert eine eindeutige Zuordnung. Hier zeigen Teilmodelltemplates Vorteile, da sie den jeweils sichtbaren Oberflächenausschnitt einschränken und damit eine robustere Registrierung ermöglichen.

Die Orange schließlich weist eine nahezu rotationssymmetrische Gestalt auf, wodurch die Registrierung in den meisten Fällen zuverlässig gelingt (vgl. [Abb. 4.15](#)). Sowohl Vollmodell- als auch Teilmodelltemplates führen hier zu vergleichbaren Ergebnissen und sind im Wesentlichen gleichwertig.

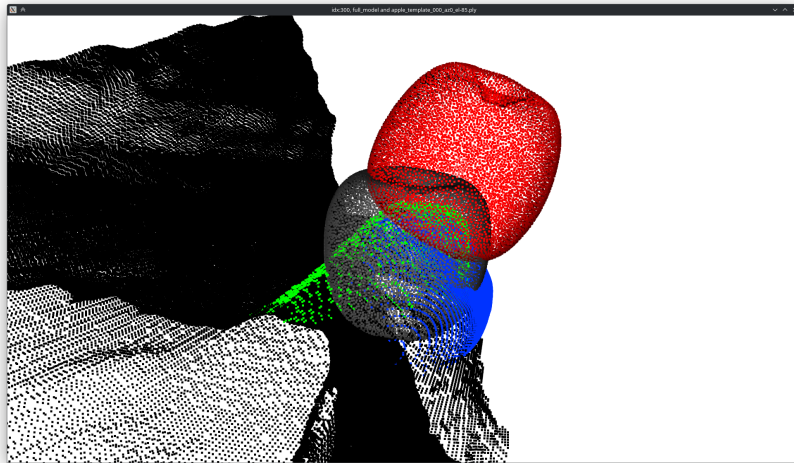


Abb. 4.12: Registrierung des Apfels (id: 300)

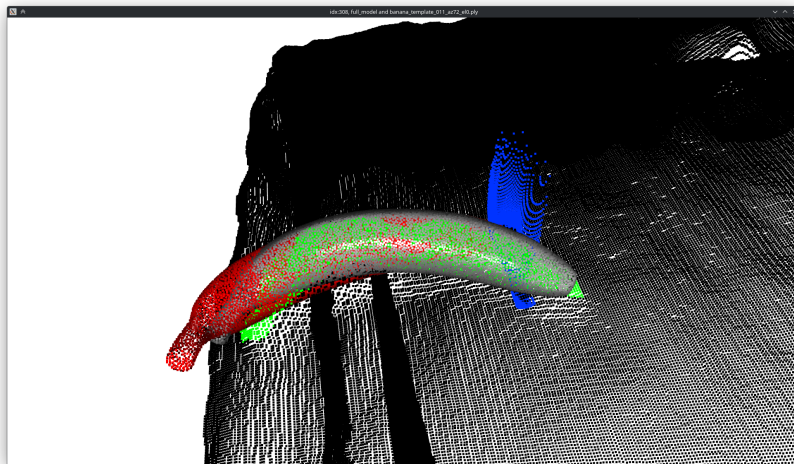


Abb. 4.13: Registrierung der Banane (id: 308)

4.4.3 Diskussion

Die Ergebnisse bestätigen die grundlegende Annahme, dass Teilmodelltemplates die Robustheit der Registrierung erhöhen können. Besonders deutlich wird dies in der tendenziell höheren Fitness, die eine bessere Übereinstimmung zwischen Modell- und Szenenpunkt-wolke widerspiegelt. Auch der im Mittel reduzierte Translationsfehler deutet darauf hin, dass die geschätzte Objektposition im Raum stabiler ermittelt wird. Demgegenüber stehen ein leicht erhöhter Inlier-RMSE sowie eine insgesamt geringere Anzahl an Inliern. Dies deutet darauf hin, dass weniger, aber dafür relevantere Korrespondenzen genutzt werden. Für die Schätzung einer initialen Pose ist dies nicht zwingend nachteilig, solange die verbleibenden Korrespondenzen hinreichend aussagekräftig sind, um eine robuste Transformation abzuleiten.

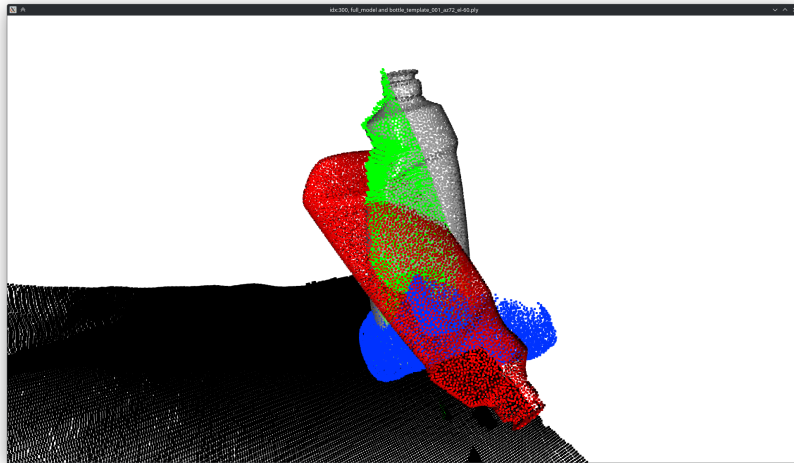


Abb. 4.14: Registrierung der Flasche (id: 308)

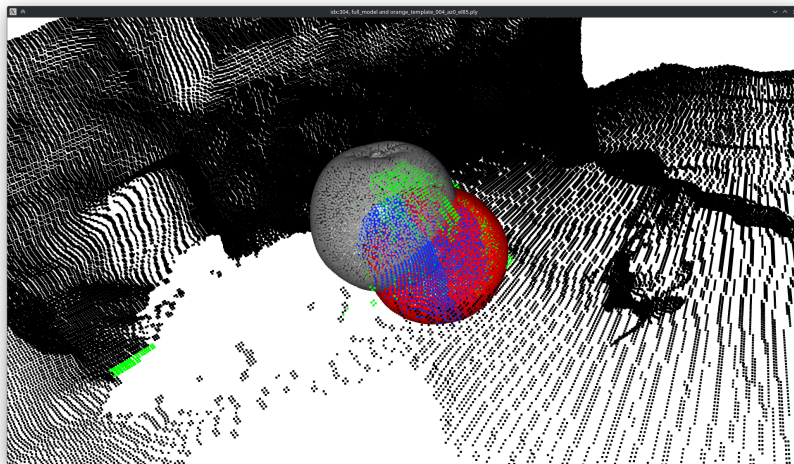


Abb. 4.15: Registrierung der Orange (id: 304)

Gleichzeitig zeigen die gelegentlich beobachteten Ausreißer im Rotationsfehler, dass die Auswahl des jeweils besten Templates nicht in allen Fällen dem globalen Optimum entspricht. Dies ist insbesondere bei Objekten mit starker Selbstähnlichkeit oder rotations-symmetrischer Geometrie kritisch, da hier Mehrdeutigkeiten bei der Ausrichtung entstehen können.

Die Übertragung der Experimente auf die Objekte Orange und Flasche verdeutlicht zudem, dass die Pipeline und die gewählte Templateanzahl nicht auf die initial untersuchten Objekte beschränkt sind, sondern auch auf weitere Formen anwendbar sind. Während bei der Orange Voll- und Teilmodelltemplates weitgehend gleichwertige Ergebnisse liefern, profitiert die Flasche deutlich stärker von Teilmodelltemplates, da diese die sichtbaren Oberflächen gezielter abbilden.

Die erhöhte Laufzeit lässt sich auf die Ergebnisse aus [Kap. 4.2.3](#) zurückführen.

Insgesamt lässt sich festhalten, dass Teilmodelltemplates einen wesentlichen Beitrag zur Beantwortung der Forschungsfrage leisten: Sie erhöhen die Robustheit der Poseinitialisierung, auch wenn dies mit einer Reduktion der Inlier-Anzahl und gelegentlichen Abweichungen im Rotationsfehler verbunden ist. Durch die Einbeziehung der visuellen Ergebnisse wird zudem deutlich, dass fehlerhafte Registrierungen nicht ausschließlich auf die Wahl des Templates zurückzuführen sind. Eine wesentliche Rolle spielt auch die Qualität der Szenenpunktwolke: Verrauschte Tiefendaten oder unvollständig rekonstruierte Oberflächen (sichtbar in den grünen Punktwolken) können zu deutlichen Abweichungen in der Registrierung führen.

Damit zeigt sich, dass Teilmodelltemplates zwar einen wichtigen Beitrag leisten, ihre Wirkung jedoch stets im Kontext von Objektgeometrie und Datenqualität zu bewerten ist. Für zukünftige Arbeiten bietet sich daher an, die Kombination von Teilmodelltemplates mit verbesserten Vorverarbeitungsmethoden oder adaptiven Template-Auswahlstrategien zu untersuchen, um die Robustheit der Registrierung weiter zu erhöhen.

4.5 Diskussion

Die durchgeführten Experimente verdeutlichen die Stärken und Grenzen des entwickelten Verfahrens zur Schätzung der initialen 6D-Pose. Im Folgenden werden die zentralen Ergebnisse zusammengeführt und kritisch reflektiert.

4.5.1 Hauptergebnisse

Die Untersuchung der Sampling-Strategien (vgl. [Kap. 4.1](#)) zeigt, dass die Wahl zwischen Uniform- und Poisson-Sampling nur geringe Auswirkungen auf die Registrierungsqualität hat. Während Poisson-Sampling erwartungsgemäß zu einer gleichmäßigeren Punktverteilung führt, unterscheiden sich die erzielten Fitness- und Fehlermetriken nur marginal. Damit eignet sich eine Modellgröße von 10 000 Punkten mit Poisson-Sampling als robuster Standard.

Die Bestimmung der Anzahl der Teilmodelltemplates für ein Objekt zeigen eine starke Abhängigkeit der Objektsymmetrie. Je komplexer das Objekt, desto mehr Ansichten werden benötigt, um es mit den Teilmodelltemplates abbilden zu können.

Die Parameteroptimierung (vgl. [Kap. 3.11](#)) und die Untersuchung der daraus abgeleiteten Universalkonfiguration (vgl. [Kap. 4.3](#)) führen zu deutlichen Verbesserungen in Fitness und Laufzeit. Auch hier gilt die Abhängigkeit der Objektgeometrie und Komplexität. Kompakte, rotationssymmetrische Objekte wie der Apfel profitieren stärker als langgestreckte Formen wie die Banane. Rotationsfehler konnten durch die Optimierung nicht wesentlich reduziert werden.

Die Verwendung von Teilmodelltemplates (vgl. [Kap. 4.4](#)) steigert die Fitness, führt jedoch nicht in allen Fällen zu präziseren Posen. Es kommt zwar zu robusteren Korrespondenzfindungen, jedoch auf Kosten von weniger Inliern, was zu lokalen Minima führen kann.

4.5.2 Kritische Reflexion

Die Experimente verdeutlichen, dass die Registrierung stark von den geometrischen Eigenschaften der Objekte abhängt. Symmetrische Strukturen führen zu Ambiguitäten in der Rotationsschätzung, wodurch die Aussagekraft der Metriken eingeschränkt ist. Für asymmetrische Objekte sind hingegen vor allem partielle Sichtbarkeit und Punktwolkenqualität limitierende Faktoren.

Zudem wurde bewusst auf Echtzeitanforderungen verzichtet, sodass die Pipeline nicht für zeitkritische Anwendungen geeignet ist. In strukturierten Szenen kann die vorgestellte Methode jedoch robuste Ergebnisse liefern.

4.5.3 Bezug zur Forschungsfrage

Die Forschungsfrage lautete, wie die initiale 6D-Pose eines Objekts mithilfe klassischer Verfahren zuverlässig geschätzt werden kann. Die Ergebnisse zeigen, dass eine Kombination aus FPFH, RANSAC und ICP eine praktikable Lösung darstellt. Die Robustheit kann durch geeignete Vorverarbeitung und Parameteroptimierung weiter gesteigert werden. Gleichzeitig wird deutlich, dass bestimmte Herausforderungen, insbesondere Symmetrien und komplexe Objektformen, mit rein klassischen Methoden nur eingeschränkt lösbar sind.

4.6 Zusammenfassung

4.6.1 Fazit

Ziel dieser Arbeit war die Entwicklung und Evaluierung einer Pipeline zur initialen Schätzung der 6D-Pose eines Objekts in einer Szene auf Basis von RGB-D-Daten mittels klassischer feature- und geometriebasierter Verfahren. Die zentrale Forschungsfrage lautete, wie gut sich die Pose auf diese Weise schätzen lässt und welchen Einfluss Teilmodelltemplates auf die Genauigkeit und Robustheit der Registrierung haben.

Die Experimente zeigen, dass eine Kombination aus FPFH, RANSAC und ICP eine zuverlässige Initialisierung ermöglicht. Die Genauigkeit hängt jedoch stark von der Objektgeometrie ab: Rotationssymmetrische Objekte wie der Apfel können bereits mit wenigen Templates robust registriert werden, während komplexere Formen wie die Banane eine feinere Abdeckung durch zusätzliche Teilmodelltemplates erfordern. Insgesamt steigern Teilmodelltemplates die Robustheit der Registrierung, indem sie sichtbare Objektteile gezielter repräsentieren, auch wenn dies mit einer reduzierten Inlier-Anzahl und gelegentlich erhöhtem Rotationsfehler einhergeht.

Darüber hinaus konnte durch eine systematische Hyperparameteroptimierung eine Universalkonfiguration ermittelt werden, die sowohl für unterschiedliche Objekte als auch für verschiedene Szenen robuste Ergebnisse liefert. Damit wurde gezeigt, dass klassische Verfahren auch ohne Deep-Learning-Ansätze eine verlässliche Poseinitialisierung ermöglichen, sofern die Parameter sorgfältig abgestimmt werden und die gewählten Templates das Objekt hinreichend abdecken.

4.6.2 Ausblick

Die Ergebnisse legen nahe, dass sich die vorgestellte Methode insbesondere für strukturierte Szenarien der robotischen Greifplanung eignet, wo robuste Initialisierungen wichtiger sind als Echtzeitfähigkeit. Für zukünftige Arbeiten ergeben sich mehrere Ansatzpunkte:

- **Verbesserte Vorregistrierung:** Integration einer initialen Orientierungsschätzung (z. B. durch PCA) könnte die Robustheit gegenüber stark rotierten Objekten erhöhen.
- **Symmetrie-bewusste Verfahren:** Neue Metriken oder Mehrhypothesen-Strategien können helfen, Ambiguitäten bei symmetrischen Objekten zu reduzieren.

- **Hybride Ansätze:** Trotzdem, dass in dieser Arbeit bewusst auf lernbasierte Verfahren verzichtet wurde, kann eine Kombination aus klassischen Methoden und lernbasierten Verfahren (z. B. für die Segmentierung oder Posen-Vorhersage) Potenzial bieten, die Pipeline weiter zu verbessern.
- **Umgang mit Objektverdeckungen:** Da reale Szenen häufig nur einen Teil des Objekts sichtbar machen, sollte die Robustheit gegenüber partiellen Daten verbessert werden. Denkbar sind Strategien wie die gezielte Generierung von Teilmodelltemplates für verdeckte Szenarien, die Verwendung redundanter Ansichten oder Verfahren zur Erkennung und Kompensation verdeckter Bereiche.
- **Echtzeitfähigkeit:** GPU-gestützte Optimierungen könnten die Laufzeit so weit reduzieren, dass der Ansatz auch für zeitkritische Anwendungen einsetzbar wird.

Insgesamt liefert diese Arbeit eine fundierte Grundlage für die initiale Pose-Schätzung mit klassischen Methoden und zeigt zugleich konkrete Wege auf, wie die Robustheit und Praktikabilität in zukünftigen Arbeiten weiter gesteigert werden können.

Literaturverzeichnis

- [1] D. Holz, S. Holzer, R. B. Rusu, and S. Behnke, “Real-time plane segmentation using rgb-d cameras,” in *Robot Soccer World Cup*. Springer, 2011, pp. 306–317.
- [2] T. Hodaň, M. Sundermeyer, B. Drost, Y. Labbé, E. Brachmann, F. Michel, C. Rother, and J. Matas, “Bop challenge 2020 on 6d object localization,” in *European Conference on Computer Vision*. Springer, 2020, pp. 577–594.
- [3] P. Henry, M. Krainin, E. Herbst, X. Ren, and D. Fox, “Rgb-d mapping: Using kinect-style depth cameras for dense 3d modeling of indoor environments,” *The international journal of Robotics Research*, vol. 31, no. 5, pp. 647–663, 2012.
- [4] H. A. Hashim, “Special orthogonal group $so(3)$, euler angles, angle-axis, rodriguez vector and unit-quaternion: Overview, mapping and challenges,” *ArXiv preprint ArXiv:1909.06669*, 2019.
- [5] Z. Zhang, “Microsoft kinect sensor and its effect,” *IEEE multimedia*, vol. 19, no. 2, pp. 4–10, 2012.
- [6] J. Han, L. Shao, D. Xu, and J. Shotton, “Enhanced computer vision with microsoft kinect sensor: A review,” *IEEE transactions on cybernetics*, vol. 43, no. 5, pp. 1318–1334, 2013.
- [7] A. Kolb, E. Barth, R. Koch, and R. Larsen, “Time-of-flight cameras in computer graphics,” in *Computer graphics forum*, vol. 29, no. 1. Wiley Online Library, 2010, pp. 141–159.
- [8] D. Scharstein and R. Szeliski, “A taxonomy and evaluation of dense two-frame stereo correspondence algorithms,” *International journal of computer vision*, vol. 47, no. 1, pp. 7–42, 2002.
- [9] R. B. Rusu, N. Blodow, and M. Beetz, “Fast point feature histograms (fpfh) for 3d registration,” in *2009 IEEE international conference on robotics and automation*. IEEE, 2009, pp. 3212–3217.

- [10] M. A. Fischler and R. C. Bolles, “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [11] P. J. Besl and N. D. McKay, “Method for registration of 3-d shapes,” in *Sensor fusion IV: control paradigms and data structures*, vol. 1611. Spie, 1992, pp. 586–606.
- [12] Y. Chen and G. Medioni, “Object modelling by registration of multiple range images,” *Image and vision computing*, vol. 10, no. 3, pp. 145–155, 1992.
- [13] D. Q. Huynh, “Metrics for 3d rotations: Comparison and analysis,” *Journal of Mathematical Imaging and Vision*, vol. 35, no. 2, pp. 155–164, 2009.
- [14] Intel® RealSense™ Tiefenkamera D435i. [Online]. Available: <https://www.intel.de/content/www/de/de/products/sku/190004/intel-realsense-depth-camera-d435i/specifications.html>
- [15] Intel® RealSense™ Technik. [Online]. Available: <https://www.intel.de/content/www/de/de/architecture-and-technology/realsense-overview.html>
- [16] Berk Calli and Arjun Singh and James Bruce and Aaron Walsman and Kurt Konolige and Siddhartha Srinivasa and Pieter Abbeel and Aaron M Dollar, “Yale-CMU-Berkeley dataset for robotic manipulation research,” *The International Journal of Robotics Research*, vol. 36, no. 3, pp. 261–268, 2017. [Online]. Available: <https://doi.org/10.1177/0278364917700714>
- [17] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A modern library for 3D data processing,” *arXiv:1801.09847*, 2018.
- [18] G. Jocher, J. Qiu, and A. Chaurasia, “Ultralytics YOLO,” Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [19] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.

Quellenverzeichnis

Abb. 2.1 Quelle: Projekt RoboTeach

Abb. 2.3 Quelle: Projekt RoboTeach

Tab. 3.2 Quelle: <https://docs.ultralytics.com/de/tasks/segment/#models>; abgerufen am 31.07.2025 um 11:44 Uhr

Abbildungsverzeichnis

2.1	Bild eines Apfels als RGB und Tiefenbild.	6
2.2	Azimut und Elevation	6
2.3	Apfel in Bounding Box vs. Segment	7
3.1	Verarbeitungspipeline zur initialen 6D-Pose-Schätzung	14
3.2	Modell des Apfels im Mesh-Format	18
3.3	Modell des Apfels als Punktwolke mit 10 000 Punkten	18
3.4	Apfel Teilmodelltemplates mit $n = 5$	20
3.5	Bananen Teiltemplates mit $n = 25$	21
4.1	Fitness in Abhängigkeit von Sampling-Methode und Punktzahl	33
4.2	Durchschnittliche Laufzeit pro Szene in Abhängigkeit von Sampling-Methode und Punktzahl	34
4.3	Apfel: Durchschnittlicher maximaler Fitnesswert vs. Anzahl der Teilmodell- templates	36
4.4	Apfel: Laufzeit vs. Anzahl der Teilmodelltemplates	36
4.5	Banane: Durchschnittlicher maximaler Fitnesswert vs. Anzahl der Teil- modelltemplates	37
4.6	Banane: Laufzeit vs. Anzahl der Teilmodelltemplates	37
4.7	Banane: Inlier-RMSE vs. Anzahl der Teilmodelltemplates	38
4.8	Banane: Trade-off zwischen Fitness und Laufzeit (gruppiert nach Höhen- ringen)	38
4.9	Fitness-Verteilung Volltemplate vs. (bestes) Teilmodelltemplate	41
4.10	Inlier-RMSE-Verteilung Volltemplate vs. (bestes) Teilmodelltemplate	42
4.11	Inlier-Anzahl Volltemplate vs. (bestes) Teilmodelltemplate	43
4.12	Registrierung des Apfels (id: 300)	44
4.13	Registrierung der Banane (id: 308)	44
4.14	Registrierung der Flasche (id: 308)	45
4.15	Registrierung der Orange (id: 304)	45

Tabellenverzeichnis

3.1	Kameraintrinsik für Intel Realsense D435i	15
3.2	Ultralytics YOLO11 Segmentation Modelle	16
3.3	Ergebnisse der Segmentierungsevaluierung mit YOLO11-Modellen.	16
A.1	Eingesetzte KI-Werkzeuge und deren Verwendung	55
A.2	Baseline Konfiguration	56
A.3	Optimale Parameter pro Objekt (gerundet).	57
A.4	Universalkonfiguration	58
A.5	Apfel: Elevationsverteilung je Template-Set (Azimut stets 0°).	58
A.6	Banane: Elevations- und Azimutverteilung je Template-Set.	59
A.7	Suchbereiche der in Optuna optimierten Parameter.	60
A.8	Mittelwerte zentraler Metriken nach Objekt und Template-Art.	60
A.9	Hardware-Spezifikationen des Testsystems	61
A.10	Software und Bibliotheken	61

A Anhang

A.1 Dokumentation der Verwendung von KI-Werkzeugen

In meiner wissenschaftlichen Arbeit habe ich KI-Werkzeuge wie im Folgenden beschrieben eingesetzt:

Tab. A.1: Eingesetzte KI-Werkzeuge und deren Verwendung

Zweck der Verwendung	Eingesetztes KI-Werkzeug	Verwendungsweise bzw. -ort
Überarbeitung von Formulierungen und Schreibstil	DeepL Write	Generierung von Vorschlägen in der gesamten Arbeit.
Erstellung einer Gliederung und Grundstruktur	ChatGPT-4	Generierung von Vorschlägen zu Beginn des Schreibprozesses.
Generierung von Verbesserungsvorschlägen für den Programmcode	Grok (xAI)	Dazugehöriges Repository (vgl. Kap. A.2)
Generierung von Programmcode sowie Verbesserungsvorschlägen dafür	ChatGPT-5	Dazugehöriges Repository (vgl. Kap. A.2)
Erstellung eines Schaubilds	ChatGPT-5	Abb. 2.2 wurde mithilfe von Matplotlib erstellt. Der Code dafür ist KI-generiert.
Formattierungshilfe	ChatGPT-5	Generierung für Formattierungsvorschläge in der gesamten Arbeit.
Rechtschreibfehlerprüfung	ChatGPT-5	Überprüfung auf Rechtschreibfehler (mit manueller Verbesserung) in der gesamten Arbeit.

A.2 Quellcode

Zur Reproduzierbarkeit und Nachvollziehbarkeit der Experimente stehen der entwickelte Quellcode und die Konfigurationen online zur Verfügung.

- **GitHub (öffentlich)** <https://github.com/JonathanWekesser/template-base-d-3d-pose-estimation>
- **GitLab (Hochschul-intern)** https://gitlab.rwu.de/stud-iki/thesis-bachelor/ws25_wekesser_3d-pose-estimation

A.3 Konfigurationen

A.3.1 Baseline Konfiguration

Alle Parameter der Baseline-Konfiguration (vgl. Tab. A.2) wurden zu Beginn der Arbeit festgelegt. Sie basieren nicht auf einer systematischen Optimierung, sondern wurden auf Grundlage explorativer Läufe mit ICP bestimmt, um eine solide Ausgangsbasis für die weiteren Experimente zu gewährleisten.

Tab. A.2: Baseline Konfiguration

Parameter	Wert
Voxelgröße [m]	0.005
Stat. Nachbarn	50
Stat. Std.-Ratio	0.7
Normalenradius [m]	0.1
Max. Normalen-NN	30
FPFH-Radius [m]	0.025
Max. FPFH-NN	100
RANSAC-Korr.-Dist. [m]	0.008
RANSAC- n	4
RANSAC-Kantenlänge	0.9
RANSAC Iterationen	100 000
RANSAC Konfidenz	0.99
ICP-Variante	p2p
ICP-Korr.-Dist. [m]	0.005
ICP Iterationen	100

A.3.2 Optimierte Konfigurationen

Die folgenden beiden Konfigurationen in [Tab. A.3](#) sind die Ergebnisse der Optuna Studien der Hyperparameter aus [Kap. 3.11](#).

Tab. A.3: Optimale Parameter pro Objekt (gerundet).

Parameter	Apfel	Banane
Voxelgröße [m]	0.020	0.019
Stat. Nachbarn	91	100
Stat. Std.-Ratio	0.202	0.117
Normalenradius [m]	0.0171	0.043
Max. Normalen-NN	78	33
FPFH-Radius [m]	0.059	0.085
Max. FPFH-NN	30	30
RANSAC-Korr.-Dist. [m]	0.005	0.005
RANSAC- n	5	3
RANSAC-Kantenlänge	0.787	0.915
RANSAC Iterationen	98 406	11 217
RANSAC Konfidenz	0.92	0.98
ICP-Variante	p2p	p2p
ICP-Korr.-Dist. [m]	0.016	0.005
ICP Iterationen	39	161
Bester Score	0.704	0.438

A.3.3 Universalkonfiguration

Die nachfolgende Universalkonfiguration fasst die in den Experimenten ermittelten Parameterwerte zusammen. Sie dient als praxisnaher Kompromiss zwischen den objektspezifischen Optima und bildet die Grundlage für die in [Kap. 4.3](#) dargestellten Auswertungen.

Es ist zu beachten, dass diese Konfiguration nicht zwingend für alle Objekte optimal ist, sondern eine robuste Baseline darstellt, die auf unterschiedliche Objekte ohne weitere Optimierung angewandt werden kann. Für spezifische Anwendungen kann eine Feinjustierung der Parameter weiterhin von Vorteil sein.

Tab. A.4: Universalkonfiguration

Parameter	Wert
Voxelgröße [m]	0.019
Stat. Nachbarn	95
Stat. Std.-Ratio	0.16
Normalenradius [m]	0.038
Max. Normalen-NN	60
FPFH-Radius [m]	0.076
Max. FPFH-NN	30
RANSAC-Korr.-Dist. [m]	0.005
RANSAC- n	4
RANSAC-Kantenlänge	0.85
RANSAC Iterationen	60 000
RANSAC Konfidenz	0.95
ICP-Variante	p2p
ICP-Korr.-Dist. [m]	0.010
ICP Iterationen	120

A.4 Verteilungen der Teilmodelltemplate-Ansichten

Tab. A.5: Apfel: Elevationsverteilung je Template-Set (Azimut stets 0°).

Set	Elevationsringe [°]
n01	{0}
n02	{-85, 85}
n03	{-85, 0, 85}
n04	{-85, -19, 19, 85}
n05	{-85, -30, 0, 30, 85}
n06	{-85, -37, -12, 12, 37, 85}
n07	{-85, -42, -19, 0, 19, 42, 85}
n08	{-85, -45, -25, -8, 8, 25, 45, 85}
n09	{-85, -48, -30, -14, 0, 14, 30, 48, 85}
n10	{-85, -51, -34, -19, -6, 6, 19, 34, 51, 85}

Tab. A.6: Banane: Elevations- und Azimutverteilung je Template-Set.

Set	Elevationsringe [°]	n/Ring	Azimut-Schritt [°]
n08_el0	{0}	8	45
n12_el0	{0}	12	30
n10_elpm30	{-30, 30}	5	72
n20_elpm30	{-30, 30}	10	36
n30_elpm30	{-30, 30}	15	24
n12_el0_pm30	{-30, 0, 30}	4	90
n18_el0_pm30	{-30, 0, 30}	6	60
n24_el0_pm30	{-30, 0, 30}	8	45
n30_el0_pm30	{-30, 0, 30}	10	36
n12_elpm15_pm45	{-45, -15, 15, 45}	3	120
n16_elpm15_pm45	{-45, -15, 15, 45}	4	90
n20_elpm15_pm45	{-45, -15, 15, 45}	5	72
n24_elpm15_pm45	{-45, -15, 15, 45}	6	60
n10_el0_pm30_pm60	{-60, -30, 0, 30, 60}	2	180
n15_el0_pm30_pm60	{-60, -30, 0, 30, 60}	3	120
n20_el0_pm30_pm60	{-60, -30, 0, 30, 60}	4	90
n25_el0_pm30_pm60	{-60, -30, 0, 30, 60}	5	72
n30_el0_pm30_pm60	{-60, -30, 0, 30, 60}	6	60

n/Ring: Anzahl der Templates pro Ring

Azimut-Schritt: resultierend aus *n/Ring*

A.5 Optuna Suchbereiche

In [Tab. A.7](#) sind die Suchbereiche der Hyperparameteroptimierung mit Optuna dokumentiert. Die mit „log“ gekennzeichneten Parameter werden logarithmisch durchsucht.

Tab. A.7: Suchbereiche der in Optuna optimierten Parameter.

Parameter	Key	Bereich	log
Voxelgröße	voxel_size	[0.002, 0.02] m	Ja
Stat. Nachbarn	stat_nb_neighbors	[10, 100]	–
Stat. Std-Ratio	stat_std_ratio	[0.1, 2.0]	Ja
Normalenradius	normal_radius	[0.01, 0.06] m	Ja
Normalen Max-NN	normal_max_nn	[20, 100]	–
FPFH-Radius	fpfh_radius	[0.01, 0.10] m	Ja
FPFH Max-NN	fpfh_max_nn	[30, 200]	–
RANSAC Corr-Dist	ransac_corr_dist	[0.003, 0.05] m	Ja
RANSAC Stichprobe	ransac_n	{3,4,5,6}	–
RANSAC Kantenprüfung	ransac_edge_length	[0.70, 0.99]	–
RANSAC Iterationen	ransac_max_iter	[4,000, 200,000]	Ja
RANSAC Konfidenz	ransac_confidence	[0.90, 0.999]	–
ICP-Variante	icp_variant	{p2p, p2l}	–
ICP Corr-Dist	icp_corr_dist	[0.001, 0.05] m	Ja
ICP Iterationen	icp_max_iter	[20, 2000]	Ja

A.6 Ergebnisse der Evaluierung der Teilmodelltemplates gegenüber Vollmodelltemplates

Hier sind die Ergebnisse der Evaluierung der Teilmodelltemplates gegenüber Vollmodelltemplates dokumentiert (s. [Tab. A.8](#)). Dabei ist das Vollmodell das Vollmodelltemplate und das Teilmodell das Teilmodelltemplate mit dem besten Ergebnis.

Tab. A.8: Mittelwerte zentraler Metriken nach Objekt und Template-Art.

Objekt	Template-Art	Fitness	Inlier-RMSE [m]	Trans.-Fehler [m]	Rot.-Fehler [°]
Apfel	Vollmodell	0.512	0.014	0.069	140.666
	Teilmodell	0.958	0.009	0.071	138.767
Banane	Vollmodell	0.231	0.013	0.051	113.287
	Teilmodell	0.865	0.011	0.096	126.309
Flasche	Vollmodell	0.228	0.013	0.073	147.328
	Teilmodell	0.641	0.012	0.126	129.258
Orange	Vollmodell	0.625	0.012	0.035	118.132
	Teilmodell	0.973	0.009	0.042	119.478

A.7 Hardware- und Software-Spezifikationen

Die Experimente wurden auf demselben System (vgl. [Tab. A.9](#)) ausgeführt, um homogene Ergebnisse zu erhalten. Die verwendete Software ist in [Tab. A.10](#) dokumentiert.

Tab. A.9: Hardware-Spezifikationen des Testsystems

Komponente	Spezifikation
Prozessor	AMD Ryzen™ 7 5800X (8 Kerne, 3.8 GHz Basistakt)
Arbeitsspeicher	32 GB DDR4 RAM (3600 MHz)
Grafikkarte	AMD Radeon™ RX 6900 XT (16 GB VRAM)
Betriebssystem	Ubuntu 24.04 LTS (64-bit)

Aufgrund von Inkompatibilitäten der verwendeten Bibliotheken mit der im System vorhandenen AMD-GPU, wurde die Grafikkarte nicht für Berechnungen eingesetzt. Sämtliche Experimente und Optimierungen wurden ausschließlich auf der CPU durchgeführt. Dies beeinflusst zwar die Laufzeit, hat jedoch keinen Einfluss auf die Korrektheit oder Reproduzierbarkeit der Ergebnisse.

Tab. A.10: Software und Bibliotheken

Software	Version
Python	3.12.10
dotenv	0.9.9
ipywidgets	8.1.7
jupyterlab	4.5.0a1
open3d	0.19.0
opencv-python	4.11.0.86
optuna	4.5.0
seaborn	0.13.2
simpleicp	2.0.15
ultralitics	8.3.146

Eidesstattliche Versicherung

Wekesser, Jonathan:

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und dass die Arbeit in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Zusatz: In der Anlage „Dokumentation der Verwendung von KI-Werkzeugen“ (Kap. A.1) habe ich dargelegt, welche KI-Werkzeuge ich genutzt habe, für welchen Zweck ich diese verwendet habe und auf welche Weise die Nutzung stattfand.

Boierfurt 17.10.2025
Ort, Datum

J. Wekesser
Unterschrift